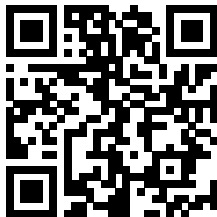
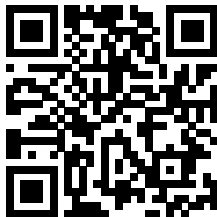


Code for the Exercises



Kindling Solver, see README.md and files in slides-proofs/:

`https://github.com/ciaranm/kindling`

VeriPB (official version), see README.md and try to run `veripb justified.opb justified.pbp`:

`https://gitlab.com/MIA0research/software/VeriPB`

VeriPB (experimental interactive mode, probably Linux and Mac only), see README.md and then use `cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml`:

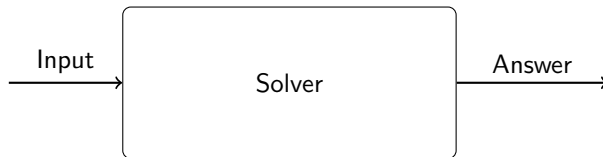
`https://github.com/ciaranm/veripb-repl`

Proof Logging for Constraint Programming

Ciaran McCreesh

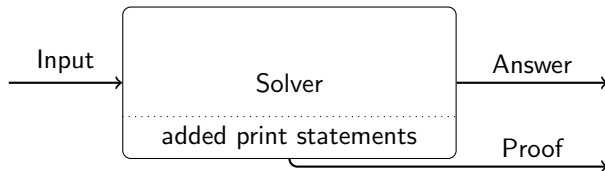


Proof Logging Recap



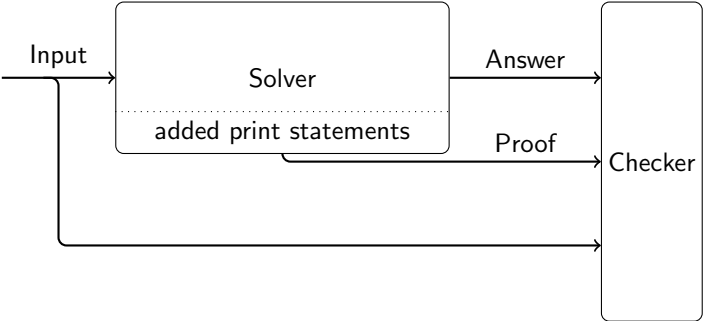
- 1 Run solver on problem input.

Proof Logging Recap



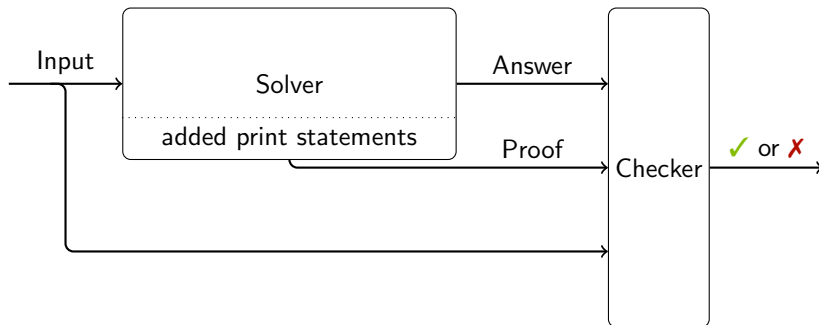
- 1 Run solver on problem input.
- 2 Solver also prints out a proof as part of its output.

Proof Logging Recap



- 1 Run solver on problem input.
- 2 Solver also prints out a proof as part of its output.
- 3 Feed input + solution + proof to proof checker.

Proof Logging Recap



- 1 Run solver on problem input.
- 2 Solver also prints out a proof as part of its output.
- 3 Feed input + solution + proof to proof checker.
- 4 Verify that proof checker says solution is correct.

The Slide That Keeps Getting Me Into Trouble

2021 **MiniZinc** challenge: for 1.28% of instances, wrong solutions were claimed.

- False claims of unsatisfiability.
- False claims of optimality.
- Infeasible solutions produced.
- Not limited to a single solver, problem, or constraint.
- Not even consistent—same solver on same hardware and same instance can give different results on different runs.

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

- **Can it say it?** The format has 0/1 variables and linear inequalities over them. We have integer variables, and a catalogue of several hundred global constraints.

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

- **Can it say it?** The format has 0/1 variables and linear inequalities over them. We have integer variables, and a catalogue of several hundred global constraints.
- **Can it prove it?** Existing is not enough, proofs have to be short. The system SAT solvers reason in needs exponentially many steps to refute the pigeonhole principle.

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

- **Can it say it?** The format has 0/1 variables and linear inequalities over them. We have integer variables, and a catalogue of several hundred global constraints.
- **Can it prove it?** Existing is not enough, proofs have to be short. The system SAT solvers reason in needs exponentially many steps to refute the pigeonhole principle.
- **Can we work it out?** A propagator is an algorithm rather than a deduction: matching, flow, dynamic programming, automata. Someone has to find the argument it is making.

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

- **Can it say it?** The format has 0/1 variables and linear inequalities over them. We have integer variables, and a catalogue of several hundred global constraints.
- **Can it prove it?** Existing is not enough, proofs have to be short. The system SAT solvers reason in needs exponentially many steps to refute the pigeonhole principle.
- **Can we work it out?** A propagator is an algorithm rather than a deduction: matching, flow, dynamic programming, automata. Someone has to find the argument it is making.
- **Can we afford it?** Nobody leaves proof logging on if it costs a hundred times as much, or if the proof is too big to check.

The Ludicrous Hypothesis: We Can Use VeriPB

A constraint programming solver can justify **everything** it does, in a proof format that knows **nothing** about constraint programming.

Four reasons this cannot possibly work:

- **Can it say it?** The format has 0/1 variables and linear inequalities over them. We have integer variables, and a catalogue of several hundred global constraints.
- **Can it prove it?** Existing is not enough, proofs have to be short. The system SAT solvers reason in needs exponentially many steps to refute the pigeonhole principle.
- **Can we work it out?** A propagator is an algorithm rather than a deduction: matching, flow, dynamic programming, automata. Someone has to find the argument it is making.
- **Can we afford it?** Nobody leaves proof logging on if it costs a hundred times as much, or if the proof is too big to check.

It mostly works. One format, one checker, and adding a constraint to your solver adds nothing to what you have to trust.

The Really Important Idea

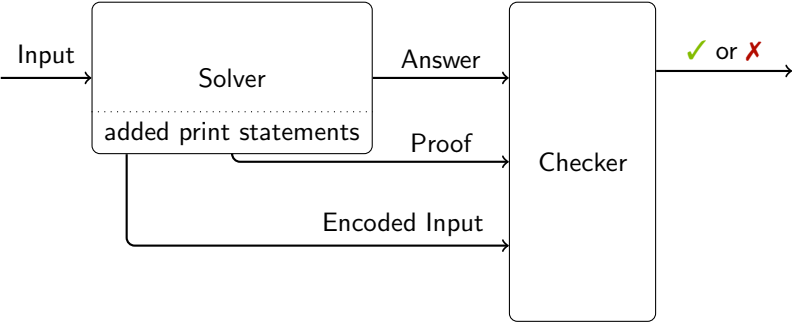
Solve exactly as normal, using the full power of constraint programming propagators etc.

Also provide a simple pseudo-Boolean **encoding** of the problem, just for proof logging.

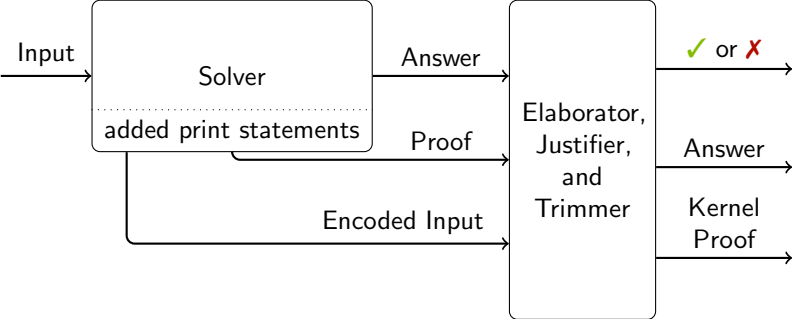
Justify the reasoning carried out against the encoding.

Critically, the solver itself does not do any reasoning about pseudo-Boolean things (unless it wants to. . .). It just prints out lightly formatted text strings.

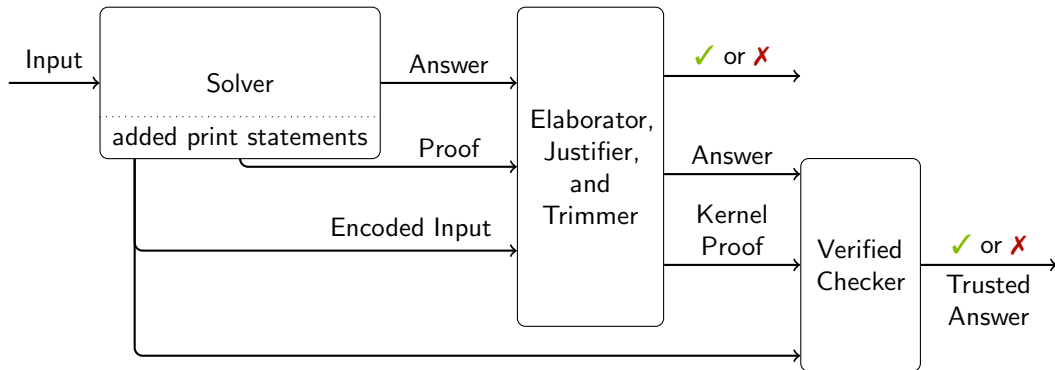
Proof Logging via Encodings



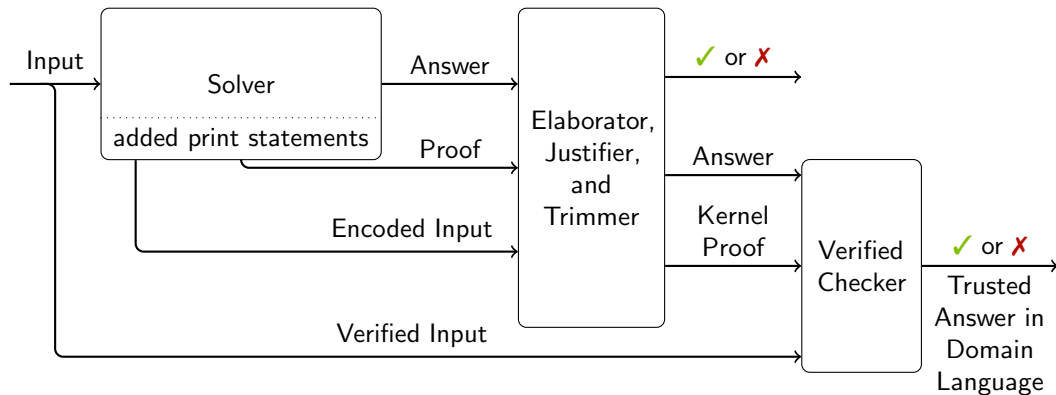
Proof Logging via Encodings



Proof Logging via Encodings



Proof Logging via Encodings



Pseudo-Boolean Notation Recap

A **pseudo-Boolean constraint** is an integer linear inequality over Booleans,

$$\sum_i c_i \ell_i \geq d$$

where a **literal** ℓ_i is a variable x or its negation $\bar{x} = 1 - x$, and a Boolean is a 1 or a 0 that we may do arithmetic with.

Throughout: upper case X, Y for constraint programming variables, lower case x, y for pseudo-Boolean literals.

A clause is the special case where every coefficient is one, and coefficients let us count:

$$x \vee \bar{y} \vee z \quad \text{is} \quad x + \bar{y} + z \geq 1 \quad \text{at most one of them} \quad \text{is} \quad \bar{x} + \bar{y} + \bar{z} \geq 2$$

VeriPB normalises everything to a $\geq$ with positive coefficients: a $\leq$ is negated, an $=$ becomes two, and $-cx$ becomes $c\bar{x} - c$. It shows you what it stored, not what you wrote:

$$1x_{b0} + 2x_{b1} + 4x_{b2} \leq 4 \quad \text{is} \quad 1\bar{x}_{b0} + 2\bar{x}_{b1} + 4\bar{x}_{b2} \geq 3$$

Pseudo-Boolean Reification Recap

$$b \Rightarrow \sum_i c_i x_i \geq d \quad \text{becomes} \quad M\bar{b} + \sum_i c_i x_i \geq d$$

for an M big enough that the constraint says nothing at all when b is false. Once everything is normalised, the smallest M that works is the degree d .

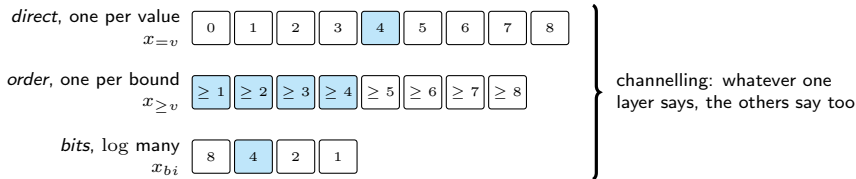
VeriPB writes it for you: $b \Rightarrow \dots$ and $b \Leftarrow \dots$ normalise to precisely that constraint. Several literals on the left of $\Rightarrow$ is a conjunction, each contributing its own $M\bar{b}_i$, and $\Leftarrow$ is just both directions—one line standing for two constraints, so it carries a label for each of them.

This is why we may write down things that are not literally pseudo-Boolean constraints:

- $x \Leftrightarrow y \wedge z$, because a conjunction **is** a pseudo-Boolean constraint, $y + z \geq 2$. Reify it in both directions and we are done.
- $f \Rightarrow X < Y$, because once X and Y are encoded, $X < Y$ is a pseudo-Boolean constraint too. Reifying it is the same rewrite again.

Encoding CP Variables

$X \in \{0 \dots 8\}$, currently taking the value 4



The bits go in the model. The order and direct literals are made in the *proof*, the first time a propagator wants one.

The Direct Encoding

An encoding has to **mean** what the variable means, and it has to provide every literal a propagator might want to talk about. It does **not** have to propagate well: this is only for checking, not solving.

For $X \in \{0 \dots 8\}$, one PB variable per value, with $x_{=v}$ meaning $X = v$:

$$x_{=0} + x_{=1} + x_{=2} + x_{=3} + x_{=4} + x_{=5} + x_{=6} + x_{=7} + x_{=8} = 1$$

- ✓ This is what a solver thinks a variable is. A domain is a set of literals, and deleting a value is falsifying one.
- ✗ One literal per value. A variable over $\{0 \dots 10^6\}$ is a million literals. Your solver might store this internally just as two integers.
- ✗ Arithmetic is clumsy. X itself is $\sum_v v \cdot x_{=v}$, so adding up three variables needs a term for every value of every one of them.

The Order Encoding

One literal per value again, but $x_{\geq v}$ meaning $X \geq v$. Nothing says so yet, so we say it:

$$x_{\geq 2} \Rightarrow x_{\geq 1}, \quad x_{\geq 3} \Rightarrow x_{\geq 2}, \quad \dots \quad x_{\geq 8} \Rightarrow x_{\geq 7}$$

each of which is the pseudo-Boolean constraint $x_{\geq v} + \bar{x}_{\geq v+1} \geq 1$.

- ✓ A bound is one literal. Most propagators reason about bounds, and now they have something to reason with.
- ✓ The direct literals come back for free, since $X = v$ is a bound from each side:

$$x_{=v} \Leftrightarrow x_{\geq v} \wedge \bar{x}_{\geq v+1}$$

- ✗ Still one literal per value, and arithmetic is still clumsy.

The Bits Encoding

Say what the variable *is*, in binary, and introduce no literals for values at all:

$$X = 1x_{b0} + 2x_{b1} + 4x_{b2} + 8x_{b3}$$

with extra constraints bounding *X*:

$$\begin{aligned} 1x_{b0} + 2x_{b1} + 4x_{b2} + 8x_{b3} &\geq 0 \\ 1x_{b0} + 2x_{b1} + 4x_{b2} + 8x_{b3} &\leq 8 \end{aligned}$$

- ✓ Logarithmic. A million values is twenty literals.
- ✓ Arithmetic is free. $X + Y - Z \leq 4$ *is* a pseudo-Boolean constraint once the bits are substituted in: one constraint, not one constraint per value.
- ✗ There is nothing to say. No literal means $X \geq 4$, or $X \neq 3$, and those are exactly what a propagator infers.

Negative domains want a sign bit, with coefficient -2^n . Not appearing in this talk for simplicity.

Hybrid Encodings

Have all three, and say how they relate:

$$x_{\geq v} \Leftrightarrow 1x_{b0} + 2x_{b1} + 4x_{b2} + 8x_{b3} \geq v$$

$$x_{=v} \Leftrightarrow x_{\geq v} \wedge \overline{x}_{\geq v+1}$$

- The **bits** say what the variable is, and carry every linear constraint.
- The **order** and **direct** literals are used for propagator inferences.
- Each of these is a **definition**. It says nothing about the problem, only about what the literals mean, so it is always available and always true.

Lazy Variable Encodings

For serious solvers, we need something both scalable and powerful. We can get the best of all worlds:

- Keep the **bits** in the model, and introduce $x_{\geq v}$ or $x_{=v}$ the first time a propagator wants to say something about v , in the **proof** rather than in the model.
- Use **extension variables**. In **VeriPB** this is the **red** command.
- Each new literal is also linked to the neighbours it turns out to have, handing the checker $x_{\geq v} \Rightarrow x_{\geq j}$ for the closest $j < v$ that exists. Again, this is in the proof, not the definition.

Why the Lazy Encoding?

All this effort goes on integer variables, so that propagators and search become easy to write.

- **It stays small.** A `var int` in **MiniZinc** has astronomical bounds. A direct or an order encoding would not survive the declaration.
- **The layers stay in step.** Anything said in one is mirrored into the others by unit propagation alone: fix some bits and the order and direct literals follow; assert an order literal and the bits follow, and so does everything that bound implies.
- So a propagator says what it means in whichever layer suits it, and nothing has to be maintained: the order chain is already unit propagation, and “takes at least one value” is a line of cutting planes rather than an axiom.
- This is a **pseudo-Boolean** property, not a Boolean one. One constraint compares a bit vector against a number, and propagates the moment a coefficient exceeds the slack. This is a big problem for CNF-based solvers.

Stated properly in Matthew McIlree’s PhD thesis, chapter 3. If anyone is really good at Lean and wants to try formalising this, please talk to me.

The Glasgow Constraint Solver

A fairly serious conventional CP solver, performance-competitive on certain classes of problems. C++, around forty constraints, and every inference every one of them makes is justified.

- Bits in the model, order and direct literals introduced lazily in the proof, exactly as on the previous slide.
- A more-or-less full set of constraints: all-different, table, regular, element, circuit, cumulative, knapsack, and so on.
- Optimisation and enumeration as well as satisfaction, so the proof has to justify the bounds and solutions too, and not only the failures.

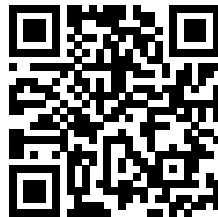


<https://github.com/ciaranm/glasgow-constraint-solver>

Kindling: a Vibe-Coded Solver, with Proof Logging, for Teaching

Under two thousand lines of Python, with no dependencies. Non-negative bounded integer variables, three constraints, depth-first search by cloning, no learning, and unsatisfiable instances only. Deliberately unpolished.

- Uses the hybrid encoding, written out in full for every variable before the search starts, no laziness.
- Written entirely by Claude—design, proofs, tests—and it steals heavily from the [Glasgow Constraint Solver](#).
- Controversial argument: you can trust it because it produces proofs.



<https://github.com/ciaranm/kindling>

Encoding CP Constraints

what the model says

$$2X + 3Y - Z \leq 12$$

$$X \neq Y$$

$$\text{table}(X_1, X_2, X_3)$$

$$\text{all-different}(X_1 \dots X_n)$$

what goes in the OPB file

→ one line: substitute the bits and it already *is* one

→ a flag f , with $f \Rightarrow X < Y$ and $\bar{f} \Rightarrow X > Y$

→ a Boolean t_i per tuple, each implying its row

→ an at-most-one per *value*, or a not-equals per *pair*

The flags and the tuple Booleans are proof-only: the solver has never heard of them, and never has to.

Linear Inequalities

Substitute the bits, and there is nothing left to do. For $X, Y, Z \in \{0 \dots 7\}$, the constraint $2X + 3Y - Z \leq 12$ is

$$\underbrace{2x_{b0} + 4x_{b1} + 8x_{b2}}_{2X} + \underbrace{3y_{b0} + 6y_{b1} + 12y_{b2}}_{3Y} - \underbrace{z_{b0} - 2z_{b1} - 4z_{b2}}_{-Z} \leq 12$$

- One PB constraint, whatever the coefficients are, and its size depends upon how many variables there are rather than upon how big their domains are.
- It propagates almost nothing, which is still fine.
- **Kindling** refuses any coefficient but ± 1 . Putting the coefficients back is one of the exercises.

Equals and Not-Equals

$X = Y$ is two inequalities, and we have just done inequalities:

$$X \leq Y \quad \text{and} \quad X \geq Y$$

$X \neq Y$ is not an inequality at all. So introduce a flag f saying which way round the two variables go, and half-reify an inequality for each way:

$$f \Rightarrow X < Y$$

$$\bar{f} \Rightarrow X > Y$$

Either way round is allowed, but f has to pick one, and neither of them is $X = Y$. For $X, Y \in \{0 \dots 3\}$, that is two pseudo-Boolean constraints:

$$4\bar{f} + 1\bar{x}_{b0} + 2\bar{x}_{b1} + 1y_{b0} + 2y_{b1} \geq 4$$

$$4f + 1x_{b0} + 2x_{b1} + 1\bar{y}_{b0} + 2\bar{y}_{b1} \geq 4$$

where the 4 is the degree: one for the strict $<$, three from the flipped signs.

Table Constraints

For $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]:$

$$\begin{aligned} t_1 + t_2 + t_3 &\geq 1 \\ t_1 &\Rightarrow x_{1=1} \wedge x_{2=2} \wedge x_{3=3} \\ t_2 &\Rightarrow x_{1=1} \wedge x_{2=3} \wedge x_{3=4} \\ t_3 &\Rightarrow x_{1=2} \wedge x_{2=2} \wedge x_{3=5} \end{aligned}$$

The tuple Booleans are proof-only. They don't (necessarily) exist inside the solver.

All-Different (Kindling Version)

One at-most-one constraint per value, over the direct literals:

for each value v :
$$x_{1=v} + x_{2=v} + \cdots + x_{n=v} \leq 1$$

- Every value gets a constraint.
- The size is the number of variables times the size of the domain, which is precisely what the last section was about avoiding.
- Easier for the exercises, though. . .

All-Different (Glasgow Version)

A clique of not-equals instead. One flag per pair, saying which way round that pair goes, and two half-reified constraints over the [bits](#):

$$s_{ij} \Rightarrow X_i < X_j$$

$$\bar{s}_{ij} \Rightarrow X_i > X_j$$

- $\binom{n}{2}$ pairs, two constraints each, and not one mention of a value anywhere. This does not grow with the domain, so an all-different over unbounded integers costs no more than one over $\{1 \dots n\}$.
- The at-most-one constraints over values have gone. They are [consequences](#) of this encoding rather than part of it.
- Can derive them in the proof, once, the first time a propagator needs one.

A Full OPB Encoding

For $X_1 \in \{0, 1\}$, $X_2, X_3, X_4 \in \{0, 1, 2\}$, all-different(X_2, X_3), all-different(X_2, X_4), all-different(X_3, X_4) with $X_1 + X_2 + X_3 + X_4 \leq 2$ and $X_2 + X_3 + X_4 \leq 1 + X_1$:

```
@x1ge1_up @x1ge1_dn x1ge1 <==> 1 x1b0 >= 1 ;
@x1eq0_up @x1eq0_dn x1eq0 <==> 1 ~x1ge1 >= 1 ;
@x1eq1_up @x1eq1_dn x1eq1 <==> 1 x1ge1 >= 1 ;

@x2_ub 2 x2b1 1 x2b0 <= 2 ;
@x2ge1_up @x2ge1_dn x2ge1 <==> 2 x2b1 1 x2b0 >= 1 ;
@x2ge2_up @x2ge2_dn x2ge2 <==> 2 x2b1 1 x2b0 >= 2 ;
@x2eq0_up @x2eq0_dn x2eq0 <==> 1 ~x2ge1 >= 1 ;
@x2eq1_up @x2eq1_dn x2eq1 <==> 1 x2ge1 1 ~x2ge2 >= 2 ;
@x2eq2_up @x2eq2_dn x2eq2 <==> 1 x2ge2 >= 1 ;

@x3_ub 2 x3b1 1 x3b0 <= 2 ;
@x3ge1_up @x3ge1_dn x3ge1 <==> 2 x3b1 1 x3b0 >= 1 ;
@x3ge2_up @x3ge2_dn x3ge2 <==> 2 x3b1 1 x3b0 >= 2 ;
@x3eq0_up @x3eq0_dn x3eq0 <==> 1 ~x3ge1 >= 1 ;
@x3eq1_up @x3eq1_dn x3eq1 <==> 1 x3ge1 1 ~x3ge2 >= 2 ;
@x3eq2_up @x3eq2_dn x3eq2 <==> 1 x3ge2 >= 1 ;

@x4_ub 2 x4b1 1 x4b0 <= 2 ;
@x4ge1_up @x4ge1_dn x4ge1 <==> 2 x4b1 1 x4b0 >= 1 ;
@x4ge2_up @x4ge2_dn x4ge2 <==> 2 x4b1 1 x4b0 >= 2 ;
@x4eq0_up @x4eq0_dn x4eq0 <==> 1 ~x4ge1 >= 1 ;
@x4eq1_up @x4eq1_dn x4eq1 <==> 1 x4ge1 1 ~x4ge2 >= 2 ;
@x4eq2_up @x4eq2_dn x4eq2 <==> 1 x4ge2 >= 1 ;
```

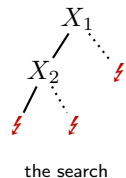
```
@amo1_0 1 x2eq0 1 x3eq0 <= 1 ;
@amo1_1 1 x2eq1 1 x3eq1 <= 1 ;
@amo1_2 1 x2eq2 1 x3eq2 <= 1 ;

@amo2_0 1 x2eq0 1 x4eq0 <= 1 ;
@amo2_1 1 x2eq1 1 x4eq1 <= 1 ;
@amo2_2 1 x2eq2 1 x4eq2 <= 1 ;

@amo3_0 1 x3eq0 1 x4eq0 <= 1 ;
@amo3_1 1 x3eq1 1 x4eq1 <= 1 ;
@amo3_2 1 x3eq2 1 x4eq2 <= 1 ;

@lin4 1 x1b0 2 x2b1 1 x2b0
      2 x3b1 1 x3b0 2 x4b1 1 x4b0 <= 2 ;
@lin5 -1 x1b0 2 x2b1 1 x2b0
      2 x3b1 1 x3b0 2 x4b1 1 x4b0 <= 1 ;
```

Proofs



prints →

```

rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ==> >= 1 ;
a    x1eq0 ==> >= 1 ; not RUP
rup >= 1 ;
conclusion UNSAT;
    
```

One **rup** line per backtrack, and one per propagation—until a propagation isn't RUP, and **a** has to assert it.

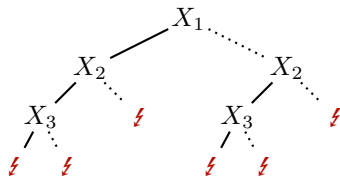
Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 $\text{all-different}(X_2, X_3)$, $\text{all-different}(X_2, X_4)$,
 $\text{all-different}(X_3, X_4)$, and

Going down the leftmost branch:

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

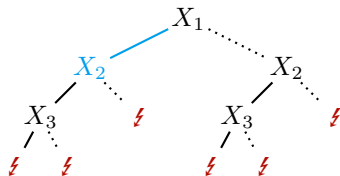
$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$

Going down the leftmost branch:

guess $X_1 = 0$

$X_2 + X_3 + X_4 \leq 1 + X_1$ now
 reads ≤ 1 , so $X_2, X_3, X_4 \leq 1$



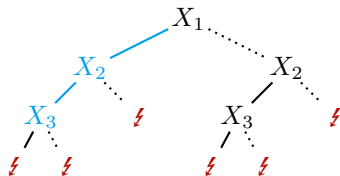
solid is the guess = 0, dotted is $\neq 0$

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

Going down the leftmost branch:

guess $X_1 = 0$

$X_2 + X_3 + X_4 \leq 1 + X_1$ now
 reads ≤ 1 , so $X_2, X_3, X_4 \leq 1$

guess $X_2 = 0$

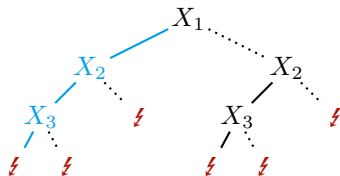
nothing: all-different here
 only looks once both of its
 variables are fixed

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

Going down the leftmost branch:

guess $X_1 = 0$

$X_2 + X_3 + X_4 \leq 1 + X_1$ now
 reads ≤ 1 , so $X_2, X_3, X_4 \leq 1$

guess $X_2 = 0$

nothing: all-different here
 only looks once both of its
 variables are fixed

guess $X_3 = 0$

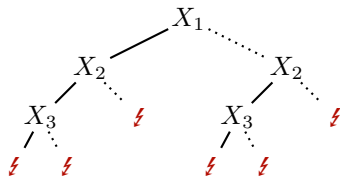
$X_2 = X_3 = 0$, so all-different
 (X_2, X_3) **fails**: backtrack

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

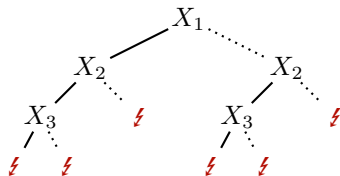
pseudo-Boolean proof version 3.0

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

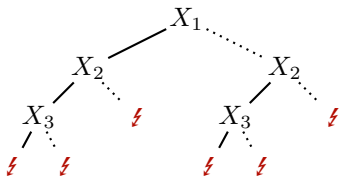
```
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

```

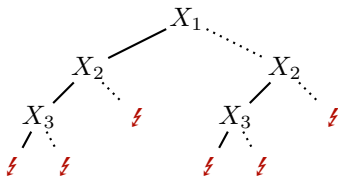
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
  
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

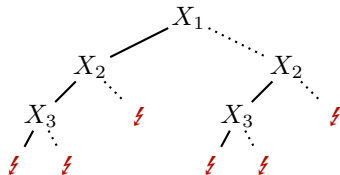
```

pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
  
```

Backtracking Search (Checking Only)

$X_1 \in \{0,1\}$ and $X_2, X_3, X_4 \in \{0,1,2\}$,
all-different(X_2, X_3), all-different(X_2, X_4),
all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$
$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

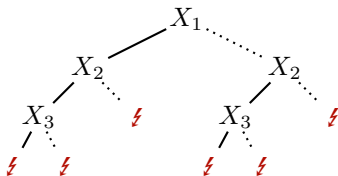
```
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ~x2eq0 ==> >= 1 ;
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

```

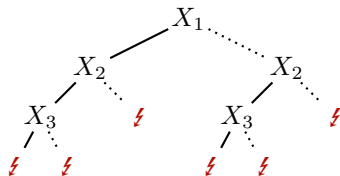
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ~x2eq0 ==> >= 1 ;
rup x1eq0 ==> >= 1 ;
  
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

```

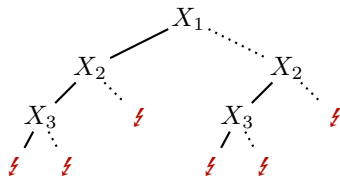
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ~x2eq0 ==> >= 1 ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ==> >= 1 ;
rup ~x1eq0 ~x2eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
  
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



solid is the guess = 0, dotted is $\neq 0$

```

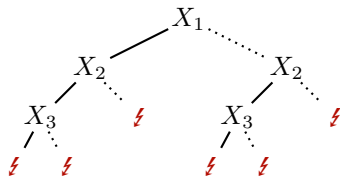
pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ~x2eq0 ==> >= 1 ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ==> >= 1 ;
rup ~x1eq0 ~x2eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
  
```

Backtracking Search (Checking Only)

$X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$,
 all-different(X_2, X_3), all-different(X_2, X_4),
 all-different(X_3, X_4), and

$$X_1 + X_2 + X_3 + X_4 \leq 2$$

$$X_2 + X_3 + X_4 \leq 1 + X_1$$



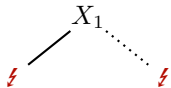
solid is the guess = 0, dotted is $\neq 0$

```

pseudo-Boolean proof version 3.0
rup x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup x1eq0 x2eq0 ==> >= 1 ;
rup x1eq0 ~x2eq0 ==> >= 1 ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ~x3eq0 ==> >= 1 ;
rup ~x1eq0 x2eq0 ==> >= 1 ;
rup ~x1eq0 ~x2eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
  
```

Constraint Propagation is Too Clever

Now say $X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$, $\text{all-different}(X_2, X_3, X_4)$, $X_1 + X_2 + X_3 + X_4 \leq 2$, $X_2 + X_3 + X_4 \leq 1 + X_1$ instead: same problem, better representation, smaller search tree.

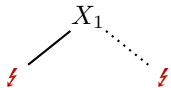


```

pseudo-Boolean proof version 3.0
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
    
```

Constraint Propagation is Too Clever

Now say $X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$, $\text{all-different}(X_2, X_3, X_4)$,
 $X_1 + X_2 + X_3 + X_4 \leq 2$, $X_2 + X_3 + X_4 \leq 1 + X_1$ instead: same problem, better representation, smaller search tree.



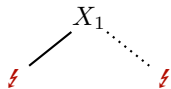
- Either guess on X_1 still leaves $X_2, X_3, X_4 \leq 1$, but now all-different sees three variables sharing only two values—a Hall set violation—and fails on the spot. X_2 and X_3 never get guessed at all.

```

pseudo-Boolean proof version 3.0
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
    
```

Constraint Propagation is Too Clever

Now say $X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$, $\text{all-different}(X_2, X_3, X_4)$,
 $X_1 + X_2 + X_3 + X_4 \leq 2$, $X_2 + X_3 + X_4 \leq 1 + X_1$ instead: same problem, better
 representation, smaller search tree.



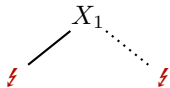
```

pseudo-Boolean proof version 3.0
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
  
```

- Either guess on X_1 still leaves $X_2, X_3, X_4 \leq 1$, but now all-different sees three variables sharing only two values—a Hall set violation—and fails on the spot. X_2 and X_3 never get guessed at all.
- Ten guesses became two, and eleven rup lines became three.

Constraint Propagation is Too Clever

Now say $X_1 \in \{0, 1\}$ and $X_2, X_3, X_4 \in \{0, 1, 2\}$, $\text{all-different}(X_2, X_3, X_4)$, $X_1 + X_2 + X_3 + X_4 \leq 2$, $X_2 + X_3 + X_4 \leq 1 + X_1$ instead: same problem, better representation, smaller search tree.



```

pseudo-Boolean proof version 3.0
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
    
```

- Either guess on X_1 still leaves $X_2, X_3, X_4 \leq 1$, but now all-different sees three variables sharing only two values—a Hall set violation—and fails on the spot. X_2 and X_3 never get guessed at all.
- Ten guesses became two, and eleven rup lines became three.
- However, this proof no longer verifies!

Justifying Propagations as Well as Backtracks?

Next attempt: put propagations in the file too. Each one claims that if the guesses hold then the bound holds, and each one claims it as **rup**:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

Justifying Propagations as Well as Backtracks?

Next attempt: put propagations in the file too. Each one claims that if the guesses hold then the bound holds, and each one claims it as **rup**:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup x1eq0 ==> >= 1 ; rejected
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

- It gets further, but that's still not enough.

Justifying Propagations as Well as Backtracks?

Next attempt: put propagations in the file too. Each one claims that if the guesses hold then the bound holds, and each one claims it as **rup**:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup x1eq0 ==> >= 1 ; rejected
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

- It gets further, but that's still not enough.
- Hall set reasoning just isn't a reverse unit property fact.

Debugging a Failed Proof

VeriPB names the line it could not get past, and `--trace-failed` says how far it got:

```
Error: Checking error at debugging.pbp:13, not implied by RUP
```

Debugging a Failed Proof

VeriPB names the line it could not get past, and `--trace-failed` says how far it got:

```
Error: Checking error at debugging.pbp:13, not implied by RUP
```

```
Propagation check failed! The propagation had the following trail:
```

```

x1eq0      (1 x1eq0 >= 1)
~x1b0      (1 x1ge1 1 ~x1b0 >= 1)
~x2b1      (1 x1b0 2 ~x2b1 1 ~x2b0 2 ~x3b1 1 ~x3b0 2 ~x4b1 1 ~x4b0 >= 8)
~x3b1 ~x4b1      (the same constraint)
~x2ge2     (2 x2b1 1 x2b0 2 ~x2ge2 >= 2)
~x3ge2 ~x4ge2     (and so on)

```

Debugging a Failed Proof

VeriPB names the line it could not get past, and `--trace-failed` says how far it got:

```
Error: Checking error at debugging.pbp:13, not implied by RUP
```

```
Propagation check failed! The propagation had the following trail:
```

```

x1eq0      (1 x1eq0 >= 1)
~x1b0      (1 x1ge1 1 ~x1b0 >= 1)
~x2b1      (1 x1b0 2 ~x2b1 1 ~x2b0 2 ~x3b1 1 ~x3b0 2 ~x4b1 1 ~x4b0 >= 8)
~x3b1 ~x4b1      (the same constraint)
~x2ge2      (2 x2b1 1 x2b0 2 ~x2ge2 >= 2)
~x3ge2 ~x4ge2      (and so on)
```

- Tells us where we made an unsupported claim, but not why. Sometimes this is easy to trace back to a cause, sometimes not so much.

Debugging a Failed Proof

VeriPB names the line it could not get past, and `--trace-failed` says how far it got:

```
Error: Checking error at debugging.pbp:13, not implied by RUP
```

```
Propagation check failed! The propagation had the following trail:
```

```
x1eq0      (1 x1eq0 >= 1)
~x1b0      (1 x1ge1 1 ~x1b0 >= 1)
~x2b1      (1 x1b0 2 ~x2b1 1 ~x2b0 2 ~x3b1 1 ~x3b0 2 ~x4b1 1 ~x4b0 >= 8)
~x3b1 ~x4b1      (the same constraint)
~x2ge2      (2 x2b1 1 x2b0 2 ~x2ge2 >= 2)
~x3ge2 ~x4ge2      (and so on)
```

- Tells us where we made an unsupported claim, but not why. Sometimes this is easy to trace back to a cause, sometimes not so much.
- The trail tells us what happened. Sometimes this is helpful, sometimes not so much.

Assertions for Propagations

The cheat: **a** adds a constraint without checking it:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
a x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
a ~x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

Assertions for Propagations

The cheat: **a** adds a constraint without checking it:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
a x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
a ~x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

- Passes UNDER ASSERTIONS, not VERIFIED: fine for development, not a complete proof.

Assertions for Propagations

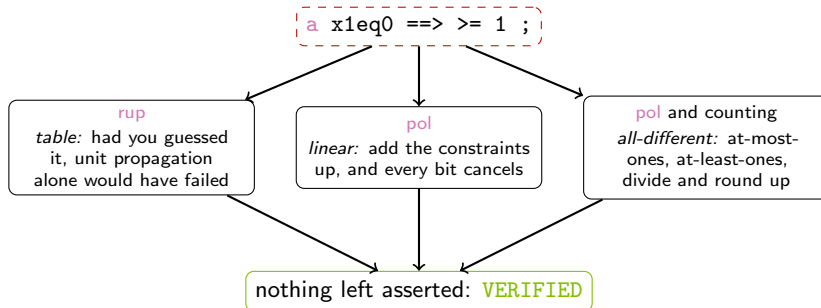
The cheat: **a** adds a constraint without checking it:

```
pseudo-Boolean proof version 3.0
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
a x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup x1eq0 ==> >= 1 ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
a ~x1eq0 ==> >= 1 : : all_different_hall
  : x2, x3, x4 in {0, 1} ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;
```

- Passes UNDER ASSERTIONS, not VERIFIED: fine for development, not a complete proof.
- The annotations are for human or external tool use.

Propagation

last section's cheat, replaced by a real derivation, one propagator at a time



Every one of them is worked out as if nothing had been guessed: the guesses above the node come along for free.

What We Have So Far

- Output a backtrack tree using **rup** steps.
- Also output propagator inferences using **rup** steps.
- When that's not enough, cheat and use assertions.

What We Have So Far

- Output a backtrack tree using **rup** steps.
- Also output propagator inferences using **rup** steps.
- When that's not enough, cheat and use assertions.
- Final step: replace these assertions with proper proof steps. We'll need to go propagator by propagator, and inference by inference.

Justifying Table Constraint Inferences: **rup** is Enough

The table from earlier, $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$, encoded again:

$$t_1 + t_2 + t_3 \geq 1$$

$$t_1 \Rightarrow x_{1=1} \wedge x_{2=2} \wedge x_{3=3}$$

$$t_2 \Rightarrow x_{1=1} \wedge x_{2=3} \wedge x_{3=4}$$

$$t_3 \Rightarrow x_{1=2} \wedge x_{2=2} \wedge x_{3=5}$$

Suppose search has guessed $X_1 = 1$. The propagator scans, finds that $X_3 = 5$ has no supporting tuple left, and deletes it.

```
rup x1eq1 ==> 1 ~x3eq5 >= 1 ;
```

Justifying Table Constraint Inferences: **rup** is Enough

The table from earlier, $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$, encoded again:

$$t_1 + t_2 + t_3 \geq 1$$

$$t_1 \Rightarrow x_{1=1} \wedge x_{2=2} \wedge x_{3=3}$$

$$t_2 \Rightarrow x_{1=1} \wedge x_{2=3} \wedge x_{3=4}$$

$$t_3 \Rightarrow x_{1=2} \wedge x_{2=2} \wedge x_{3=5}$$

Suppose search has guessed $X_1 = 1$. The propagator scans, finds that $X_3 = 5$ has no supporting tuple left, and deletes it.

```
rup x1eq1 ==> 1 ~x3eq5 >= 1 ;
```

- The checker assumes the opposite, so $X_1 = 1$ and $X_3 = 5$ both hold.

Justifying Table Constraint Inferences: **rup** is Enough

The table from earlier, $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$, encoded again:

$$t_1 + t_2 + t_3 \geq 1$$

$$t_1 \Rightarrow x_{1=1} \wedge x_{2=2} \wedge x_{3=3}$$

$$t_2 \Rightarrow x_{1=1} \wedge x_{2=3} \wedge x_{3=4}$$

$$t_3 \Rightarrow \textcolor{red}{x}_{1=2} \wedge x_{2=2} \wedge x_{3=5}$$

Suppose search has guessed $X_1 = 1$. The propagator scans, finds that $X_3 = 5$ has no supporting tuple left, and deletes it.

```
rup x1eq1 ==> 1 ~x3eq5 >= 1 ;
```

- The checker assumes the opposite, so $X_1 = 1$ and $X_3 = 5$ both hold.
- $x_{1=1}$ is $x_{1 \geq 1} \wedge \bar{x}_{1 \geq 2}$ and $x_{1=2}$ is $x_{1 \geq 2}$, so $x_{1=2}$ is false and t_3 falls over.

Justifying Table Constraint Inferences: **rup** is Enough

The table from earlier, $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$, encoded again:

$$t_1 + t_2 + t_3 \geq 1$$

$$t_1 \Rightarrow x_{1=1} \wedge x_{2=2} \wedge \textcolor{red}{x_{3=3}}$$

$$t_2 \Rightarrow x_{1=1} \wedge x_{2=3} \wedge \textcolor{red}{x_{3=4}}$$

$$t_3 \Rightarrow \textcolor{red}{x_{1=2}} \wedge x_{2=2} \wedge x_{3=5}$$

Suppose search has guessed $X_1 = 1$. The propagator scans, finds that $X_3 = 5$ has no supporting tuple left, and deletes it.

```
rup x1eq1 ==> 1 ~x3eq5 >= 1 ;
```

- The checker assumes the opposite, so $X_1 = 1$ and $X_3 = 5$ both hold.
- $x_{1=1}$ is $x_{1 \geq 1} \wedge \bar{x}_{1 \geq 2}$ and $x_{1=2}$ is $x_{1 \geq 2}$, so $x_{1=2}$ is false and t_3 falls over.
- $x_{3=5}$ gives $x_{3 \geq 5}$, which implies $x_{3 \geq 4}$: that kills $x_{3=3}$ and $x_{3=4}$, and hence t_1 and t_2 .

Justifying Table Constraint Inferences: **rup** is Enough

The table from earlier, $(X_1, X_2, X_3) \in [(1, 2, 3), (1, 3, 4), (2, 2, 5)]$, encoded again:

$$t_1 + t_2 + t_3 \geq 1$$

$$t_1 \Rightarrow x_{1=1} \wedge x_{2=2} \wedge \textcolor{red}{x_{3=3}}$$

$$t_2 \Rightarrow x_{1=1} \wedge x_{2=3} \wedge \textcolor{red}{x_{3=4}}$$

$$t_3 \Rightarrow \textcolor{red}{x_{1=2}} \wedge x_{2=2} \wedge x_{3=5}$$

Suppose search has guessed $X_1 = 1$. The propagator scans, finds that $X_3 = 5$ has no supporting tuple left, and deletes it.

```
rup x1eq1 ==> 1 ~x3eq5 >= 1 ;
```

- The checker assumes the opposite, so $X_1 = 1$ and $X_3 = 5$ both hold.
- $x_{1=1}$ is $x_{1 \geq 1} \wedge \bar{x}_{1 \geq 2}$ and $x_{1=2}$ is $x_{1 \geq 2}$, so $x_{1=2}$ is false and t_3 falls over.
- $x_{3=5}$ gives $x_{3 \geq 5}$, which implies $x_{3 \geq 4}$: that kills $x_{3=3}$ and $x_{3=4}$, and hence t_1 and t_2 .
- Nothing is left to satisfy $t_1 + t_2 + t_3 \geq 1$.

Justifying Linear Inequalities using **pol**

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

$$\text{lin1} \quad 4\bar{x}_{1b2} \quad 2\bar{x}_{1b1} \quad 1\bar{x}_{1b0} \quad 4\bar{x}_{2b2} \quad 2\bar{x}_{2b1} \quad 1\bar{x}_{2b0} \quad 4\bar{x}_{3b2} \quad 2\bar{x}_{3b1} \quad 1\bar{x}_{3b0} \quad \geq 12$$

Justifying Linear Inequalities using **pol**

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	$4\bar{x}_{1b2}$	$2\bar{x}_{1b1}$	$1\bar{x}_{1b0}$		$4\bar{x}_{2b2}$	$2\bar{x}_{2b1}$	$1\bar{x}_{2b0}$		$4\bar{x}_{3b2}$	$2\bar{x}_{3b1}$	$1\bar{x}_{3b0}$	≥ 12
@x2ge3-up					$4x_{2b2}$	$2x_{2b1}$	$1x_{2b0}$	$3\bar{x}_{2\geq 3}$				≥ 3

Justifying Linear Inequalities using **pol**

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	$4\bar{x}_{1b2}$	$2\bar{x}_{1b1}$	$1\bar{x}_{1b0}$		$4\bar{x}_{2b2}$	$2\bar{x}_{2b1}$	$1\bar{x}_{2b0}$		$4\bar{x}_{3b2}$	$2\bar{x}_{3b1}$	$1\bar{x}_{3b0}$	≥ 12
@x2ge3-up					$4x_{2b2}$	$2x_{2b1}$	$1x_{2b0}$	$3\bar{x}_{2\geq 3}$				≥ 3
@x3ge3-up									$4x_{3b2}$	$2x_{3b1}$	$1x_{3b0}$	$3\bar{x}_{3\geq 3}$
												≥ 3

Justifying Linear Inequalities using **pol**

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	$4\bar{x}_{1b2}$	$2\bar{x}_{1b1}$	$1\bar{x}_{1b0}$	$4\bar{x}_{2b2}$	$2\bar{x}_{2b1}$	$1\bar{x}_{2b0}$	$4\bar{x}_{3b2}$	$2\bar{x}_{3b1}$	$1\bar{x}_{3b0}$	≥ 12
@x2ge3_up				$4x_{2b2}$	$2x_{2b1}$	$1x_{2b0}$	$3\bar{x}_{2\geq 3}$			≥ 3
@x3ge3_up							$4x_{3b2}$	$2x_{3b1}$	$1x_{3b0}$	≥ 3
@x1ge4_up	$4x_{1b2}$	$2x_{1b1}$	$1x_{1b0}$	$4\bar{x}_{1\geq 4}$						≥ 4

Justifying Linear Inequalities using pol

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	$4\bar{x}_{1b2}$	$2\bar{x}_{1b1}$	$1\bar{x}_{1b0}$		$4\bar{x}_{2b2}$	$2\bar{x}_{2b1}$	$1\bar{x}_{2b0}$		$4\bar{x}_{3b2}$	$2\bar{x}_{3b1}$	$1\bar{x}_{3b0}$	≥ 12	
@x2ge3_up					$4x_{2b2}$	$2x_{2b1}$	$1x_{2b0}$	$3\bar{x}_{2\geq 3}$				≥ 3	
@x3ge3_up									$4x_{3b2}$	$2x_{3b1}$	$1x_{3b0}$	$3\bar{x}_{3\geq 3}$	≥ 3
@x1ge4_up	$4x_{1b2}$	$2x_{1b1}$	$1x_{1b0}$	$4\bar{x}_{1\geq 4}$								≥ 4	
sum	4	2	1	$4\bar{x}_{1\geq 4}$	4	2	1	$3\bar{x}_{2\geq 3}$	4	2	1	$3\bar{x}_{3\geq 3}$	≥ 22

Each column is $cx + c\bar{x} = c$, so the bits leave 21 behind and only 1 of the degree survives.

Justifying Linear Inequalities using **pol**

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	$4\bar{x}_{1b2}$	$2\bar{x}_{1b1}$	$1\bar{x}_{1b0}$		$4\bar{x}_{2b2}$	$2\bar{x}_{2b1}$	$1\bar{x}_{2b0}$		$4\bar{x}_{3b2}$	$2\bar{x}_{3b1}$	$1\bar{x}_{3b0}$		≥ 12
@x2ge3-up					$4x_{2b2}$	$2x_{2b1}$	$1x_{2b0}$	$3\bar{x}_{2\geq 3}$					≥ 3
@x3ge3-up									$4x_{3b2}$	$2x_{3b1}$	$1x_{3b0}$	$3\bar{x}_{3\geq 3}$	≥ 3
@x1ge4-up	$4x_{1b2}$	$2x_{1b1}$	$1x_{1b0}$	$4\bar{x}_{1\geq 4}$									≥ 4
sum	4	2	1	$4\bar{x}_{1\geq 4}$	4	2	1	$3\bar{x}_{2\geq 3}$	4	2	1	$3\bar{x}_{3\geq 3}$	≥ 22
i.e.				$4\bar{x}_{1\geq 4}$				$3\bar{x}_{2\geq 3}$				$3\bar{x}_{3\geq 3}$	≥ 1

Each column is $cx + c\bar{x} = c$, so the bits leave 21 behind and only 1 of the degree survives.

Justifying Linear Inequalities using pol

$X_1 + X_2 + X_3 \leq 9$, and something has already established $X_2 \geq 3$ and $X_3 \geq 3$. So $X_1 \leq 3$.

@lin1	4x _{1b2}	2x _{1b1}	1x _{1b0}		4x _{2b2}	2x _{2b1}	1x _{2b0}		4x _{3b2}	2x _{3b1}	1x _{3b0}		≥ 12
@x2ge3_up					4x _{2b2}	2x _{2b1}	1x _{2b0}	3x ₂ ≥3					≥ 3
@x3ge3_up									4x _{3b2}	2x _{3b1}	1x _{3b0}	3x ₃ ≥3	≥ 3
@x1ge4_up	4x _{1b2}	2x _{1b1}	1x _{1b0}	4x ₁ ≥4									≥ 4
sum	4	2	1	4x ₁ ≥4	4	2	1	3x ₂ ≥3	4	2	1	3x ₃ ≥3	≥ 22
i.e.				4x ₁ ≥4				3x ₂ ≥3				3x ₃ ≥3	≥ 1

Each column is $cx + c\bar{x} = c$, so the bits leave 21 behind and only 1 of the degree survives.

```
pol @lin1 @x2ge3_up + @x3ge3_up + @x1ge4_up + ;
rup x2ge3 x3ge3 ==> 1 ~x1ge4 >= 1 ;
```

Now the rup works: assume all three literals, and what is left reads $0 \geq 1$. In our search example we got away with using rup alone; we were lucky, and usually this doesn't work.

Justifying All-Different Failures

$$\begin{aligned} V &\in \{1 \quad \quad 4 \quad 5\} \\ W &\in \{1 \quad 2 \quad 3 \quad \quad \} \\ X &\in \{ \quad 2 \quad 3 \quad \quad \} \\ Y &\in \{1 \quad \quad 3 \quad \quad \} \\ Z &\in \{1 \quad \quad 3 \quad \quad \} \end{aligned}$$

Justifying All-Different Failures

$$\begin{aligned}
 V &\in \{1 \quad \quad 4 \quad 5\} \\
 W &\in \{1 \quad 2 \quad 3 \quad \quad \} \\
 X &\in \{ \quad 2 \quad 3 \quad \quad \} \\
 Y &\in \{1 \quad \quad 3 \quad \quad \} \\
 Z &\in \{1 \quad \quad 3 \quad \quad \}
 \end{aligned}$$

Justifying All-Different Failures

$$\begin{array}{l}
 V \in \{1 \quad \quad 4 \quad 5\} \\
 W \in \{1 \quad 2 \quad 3 \quad \quad \} \\
 X \in \{ \quad 2 \quad 3 \quad \quad \} \\
 Y \in \{1 \quad \quad 3 \quad \quad \} \\
 Z \in \{1 \quad \quad 3 \quad \quad \}
 \end{array}$$

$$w_{=1} + w_{=2} + w_{=3} \geq 1 \text{ \% } W \text{ takes some value}$$

Justifying All-Different Failures

$$\begin{array}{ll}
 V \in \{1 & 4 \ 5\} \\
 W \in \{1 \ 2 \ 3 & \} \\
 X \in \{ & 2 \ 3 \} \\
 Y \in \{1 & 3 \} \\
 Z \in \{1 & 3 \}
 \end{array}
 \begin{array}{ll}
 w_{=1} + & w_{=2} + \ w_{=3} \\
 & x_{=2} + \ x_{=3} \\
 y_{=1} & + \ y_{=3} \\
 z_{=1} & + \ z_{=3}
 \end{array}
 \begin{array}{l}
 \geq \ 1 \ \% \ W \text{ takes some value} \\
 \geq \ 1 \ \% \ X \text{ takes some value} \\
 \geq \ 1 \ \% \ Y \text{ takes some value} \\
 \geq \ 1 \ \% \ Z \text{ takes some value}
 \end{array}$$

Justifying All-Different Failures

$$\begin{array}{llll}
 V \in \{1 & & 4 & 5\} \\
 W \in \{1 & 2 & 3 & \} \quad w_{=1} + \quad w_{=2} + \quad w_{=3} & \geq 1 \text{ \% } W \text{ takes some value} \\
 X \in \{ & 2 & 3 & \} \quad \quad \quad x_{=2} + \quad x_{=3} & \geq 1 \text{ \% } X \text{ takes some value} \\
 Y \in \{1 & & 3 & \} \quad y_{=1} \quad \quad + \quad y_{=3} & \geq 1 \text{ \% } Y \text{ takes some value} \\
 Z \in \{1 & & 3 & \} \quad z_{=1} \quad \quad + \quad z_{=3} & \geq 1 \text{ \% } Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + \quad \quad \quad -y_{=1} + -z_{=1} \geq -1 \text{ \% At most one variable} = 1 \\
 \rightarrow & \quad \quad \quad -w_{=2} + -x_{=2} \geq -1 \text{ \% At most one variable} = 2 \\
 \rightarrow & \quad \quad \quad -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \text{ \% At most one variable} = 3
 \end{array}$$

Justifying All-Different Failures

$$\begin{array}{llll}
 V \in \{1 & & 4 & 5\} \\
 W \in \{1 & 2 & 3 & \} \\
 X \in \{ & 2 & 3 & \} \\
 Y \in \{1 & & 3 & \} \\
 Z \in \{1 & & 3 & \}
 \end{array}
 \begin{array}{llll}
 w_{=1} + & w_{=2} + & w_{=3} & \\
 & x_{=2} + & x_{=3} & \\
 y_{=1} & + & y_{=3} & \\
 z_{=1} & + & z_{=3} &
 \end{array}
 \begin{array}{ll}
 \geq 1 \% W \text{ takes some value} \\
 \geq 1 \% X \text{ takes some value} \\
 \geq 1 \% Y \text{ takes some value} \\
 \geq 1 \% Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + & -y_{=1} + -z_{=1} \geq -1 \% \text{ At most one variable} = 1 \\
 \rightarrow & & -w_{=2} + -x_{=2} \geq -1 \% \text{ At most one variable} = 2 \\
 \rightarrow & & -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \% \text{ At most one variable} = 3 \\
 & & -v_{=1} \geq 1 \% \text{ Sum all constraints so far}
 \end{array}$$

Justifying All-Different Failures

$$\begin{array}{llll}
 V \in \{1 & & 4 & 5\} \\
 W \in \{1 & 2 & 3 & \} \\
 X \in \{ & 2 & 3 & \} \\
 Y \in \{1 & & 3 & \} \\
 Z \in \{1 & & 3 & \}
 \end{array}
 \begin{array}{llll}
 w_{=1} + & w_{=2} + & w_{=3} & \\
 & x_{=2} + & x_{=3} & \\
 y_{=1} & + & y_{=3} & \\
 z_{=1} & + & z_{=3} &
 \end{array}
 \begin{array}{ll}
 \geq 1 \% W \text{ takes some value} \\
 \geq 1 \% X \text{ takes some value} \\
 \geq 1 \% Y \text{ takes some value} \\
 \geq 1 \% Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + & -y_{=1} + -z_{=1} \geq -1 \% \text{ At most one variable} = 1 \\
 \rightarrow & & -w_{=2} + -x_{=2} \geq -1 \% \text{ At most one variable} = 2 \\
 \rightarrow & & -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \% \text{ At most one variable} = 3
 \end{array}$$

$$\begin{array}{ll}
 -v_{=1} & \geq 1 \% \text{ Sum all constraints so far} \\
 v_{=1} & \geq 0 \% \text{ Variable } v_{=1} \text{ non-negative}
 \end{array}$$

Justifying All-Different Failures

$$\begin{array}{llll}
 V \in \{1 & & 4 & 5\} \\
 W \in \{1 & 2 & 3 & \} \\
 X \in \{ & 2 & 3 & \} \\
 Y \in \{1 & & 3 & \} \\
 Z \in \{1 & & 3 & \}
 \end{array}
 \begin{array}{llll}
 w_{=1} + & w_{=2} + & w_{=3} & \\
 & x_{=2} + & x_{=3} & \\
 y_{=1} & + & y_{=3} & \\
 z_{=1} & + & z_{=3} &
 \end{array}
 \begin{array}{ll}
 \geq 1 \% W \text{ takes some value} \\
 \geq 1 \% X \text{ takes some value} \\
 \geq 1 \% Y \text{ takes some value} \\
 \geq 1 \% Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + & -y_{=1} + -z_{=1} \geq -1 \% \text{ At most one variable} = 1 \\
 \rightarrow & & -w_{=2} + -x_{=2} \geq -1 \% \text{ At most one variable} = 2 \\
 \rightarrow & & -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \% \text{ At most one variable} = 3
 \end{array}$$

$$\begin{array}{ll}
 -v_{=1} & \geq 1 \% \text{ Sum all constraints so far} \\
 v_{=1} & \geq 0 \% \text{ Variable } v_{=1} \text{ non-negative}
 \end{array}$$

$$0 \geq 1 \% \text{ Sum above two constraints}$$

A Complete Proof, With No Cheating

```

pseudo-Boolean proof version 3.0
@atleast2 pol @x2eq0_dn @x2eq1_dn + @x2eq2_dn + ;
@atleast3 pol @x3eq0_dn @x3eq1_dn + @x3eq2_dn + ;
@atleast4 pol @x4eq0_dn @x4eq1_dn + @x4eq2_dn + ;
pol @lin3 @x1ge1_dn + @x2ge2_up + s ;
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x3ge2_up + s ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x4ge2_up + s ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup x1eq0 ==> >= 1 ;
pol @lin2 @x1ge1_up + @x2ge2_up + s ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x3ge2_up + s ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x4ge2_up + s ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;

```

A Complete Proof, With No Cheating

```

pseudo-Boolean proof version 3.0
@atleast2 pol @x2eq0_dn @x2eq1_dn + @x2eq2_dn + ;
@atleast3 pol @x3eq0_dn @x3eq1_dn + @x3eq2_dn + ;
@atleast4 pol @x4eq0_dn @x4eq1_dn + @x4eq2_dn + ;
pol @lin3 @x1ge1_dn + @x2ge2_up + s ;
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x3ge2_up + s ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x4ge2_up + s ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup x1eq0 ==> >= 1 ;
pol @lin2 @x1ge1_up + @x2ge2_up + s ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x3ge2_up + s ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x4ge2_up + s ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;

```

- The two **a** lines have become **pol** lines: the derivation from two slides ago, over at-least-ones that say what the literals mean rather than what the model said, and so are derived once at the top and used twice.

A Complete Proof, With No Cheating

```

pseudo-Boolean proof version 3.0
@atleast2 pol @x2eq0_dn @x2eq1_dn + @x2eq2_dn + ;
@atleast3 pol @x3eq0_dn @x3eq1_dn + @x3eq2_dn + ;
@atleast4 pol @x4eq0_dn @x4eq1_dn + @x4eq2_dn + ;
pol @lin3 @x1ge1_dn + @x2ge2_up + s ;
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x3ge2_up + s ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x4ge2_up + s ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup x1eq0 ==> >= 1 ;
pol @lin2 @x1ge1_up + @x2ge2_up + s ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x3ge2_up + s ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x4ge2_up + s ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;

```

- The two **a** lines have become **pol** lines: the derivation from two slides ago, over at-least-ones that say what the literals mean rather than what the model said, and so are derived once at the top and used twice.
- Six more **pol** lines are new. We had been claiming linear inferences through **rup** only and getting away with it.

A Complete Proof, With No Cheating

```

pseudo-Boolean proof version 3.0
@atleast2 pol @x2eq0_dn @x2eq1_dn + @x2eq2_dn + ;
@atleast3 pol @x3eq0_dn @x3eq1_dn + @x3eq2_dn + ;
@atleast4 pol @x4eq0_dn @x4eq1_dn + @x4eq2_dn + ;
pol @lin3 @x1ge1_dn + @x2ge2_up + s ;
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x3ge2_up + s ;
rup x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin3 @x1ge1_dn + @x4ge2_up + s ;
rup x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup x1eq0 ==> >= 1 ;
pol @lin2 @x1ge1_up + @x2ge2_up + s ;
rup ~x1eq0 ==> 1 ~x2ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x3ge2_up + s ;
rup ~x1eq0 ==> 1 ~x3ge2 >= 1 ;
pol @lin2 @x1ge1_up + @x4ge2_up + s ;
rup ~x1eq0 ==> 1 ~x4ge2 >= 1 ;
pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
rup ~x1eq0 ==> >= 1 ;
rup >= 1 ;
output NONE;
conclusion UNSAT;
end pseudo-Boolean proof;

```

- The two **a** lines have become **pol** lines: the derivation from two slides ago, over at-least-ones that say what the literals mean rather than what the model said, and so are derived once at the top and used twice.
- Six more **pol** lines are new. We had been claiming linear inferences through **rup** only and getting away with it.
- **VERIFIED**.

Recovering At-Most-One

Glasgow has no at-most-one constraint to add up: its all-different is a clique of not-equals. So we need to derive them.

Recovering At-Most-One

Glasgow has no at-most-one constraint to add up: its all-different is a clique of not-equals. So we need to derive them.

- One pair first. Assume both take v : the two half-reified constraints for that pair become unit on its selector, one each way, and contradict. So $\bar{a} + \bar{b} \geq 1$ is derivable by **rup**.

Recovering At-Most-One

Glasgow has no at-most-one constraint to add up: its all-different is a clique of not-equals. So we need to derive them.

- One pair first. Assume both take v : the two half-reified constraints for that pair become unit on its selector, one each way, and contradict. So $\bar{a} + \bar{b} \geq 1$ is derivable by **rup**.

$$\begin{aligned}
 (\bar{a} + \bar{b} \geq 1) + (\bar{a} + \bar{c} \geq 1) &= 2\bar{a} + \bar{b} + \bar{c} \geq 2 \\
 + (\bar{b} + \bar{c} \geq 1) &= 2\bar{a} + 2\bar{b} + 2\bar{c} \geq 3 \\
 \div 2 &= \bar{a} + \bar{b} + \bar{c} \geq 2
 \end{aligned}$$

Recovering At-Most-One

Glasgow has no at-most-one constraint to add up: its all-different is a clique of not-equals. So we need to derive them.

- One pair first. Assume both take v : the two half-reified constraints for that pair become unit on its selector, one each way, and contradict. So $\bar{a} + \bar{b} \geq 1$ is derivable by **rup**.

$$\begin{aligned}
 (\bar{a} + \bar{b} \geq 1) + (\bar{a} + \bar{c} \geq 1) &= 2\bar{a} + \bar{b} + \bar{c} \geq 2 \\
 + (\bar{b} + \bar{c} \geq 1) &= 2\bar{a} + 2\bar{b} + 2\bar{c} \geq 3 \\
 \div 2 &= \bar{a} + \bar{b} + \bar{c} \geq 2
 \end{aligned}$$

- Division **rounds up**, so $3/2$ becomes 2.

Recovering At-Most-One

Glasgow has no at-most-one constraint to add up: its all-different is a clique of not-equals. So we need to derive them.

- One pair first. Assume both take v : the two half-reified constraints for that pair become unit on its selector, one each way, and contradict. So $\bar{a} + \bar{b} \geq 1$ is derivable by **rup**.

$$\begin{aligned}
 (\bar{a} + \bar{b} \geq 1) + (\bar{a} + \bar{c} \geq 1) &= 2\bar{a} + \bar{b} + \bar{c} \geq 2 \\
 + (\bar{b} + \bar{c} \geq 1) &= 2\bar{a} + 2\bar{b} + 2\bar{c} \geq 3 \\
 \div 2 &= \bar{a} + \bar{b} + \bar{c} \geq 2
 \end{aligned}$$

- Division **rounds up**, so $3/2$ becomes 2.
- It generalises to n literals as one **pol** of $O(n^2)$ steps. Can then run the previous slide's argument.

Assumptions Don't Matter

Every derivation in this section was worked out as if nothing had been guessed. Inside the tree, everything is conditional on the guesses above it, but it turns out this isn't an issue.

```

rup 1 ~x2ge2 >= 1 ;      if the guess were part of the problem
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;  what we actually write

```

Matthew McIlree's thesis again, chapter 2, where the division and saturation are done properly.

Assumptions Don't Matter

Every derivation in this section was worked out as if nothing had been guessed. Inside the tree, everything is conditional on the guesses above it, but it turns out this isn't an issue.

```

rup 1 ~x2ge2 >= 1 ;      if the guess were part of the problem
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;  what we actually write

```

- **Whatever we can prove without assumptions we can prove with them**, using the same rules in the same order, for a constant amount of extra work per step.

Matthew McIlree's thesis again, chapter 2, where the division and saturation are done properly.

Assumptions Don't Matter

Every derivation in this section was worked out as if nothing had been guessed. Inside the tree, everything is conditional on the guesses above it, but it turns out this isn't an issue.

```

rup 1 ~x2ge2 >= 1 ;      if the guess were part of the problem
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;    what we actually write

```

- Whatever we can prove without assumptions we can prove with them, using the same rules in the same order, for a constant amount of extra work per step.
- For **rup**, put the assumptions on the line. Negating the implication makes the checker set every one of them, leaving exactly the problem we knew how to refute.

Matthew McIlree's thesis again, chapter 2, where the division and saturation are done properly.

Assumptions Don't Matter

Every derivation in this section was worked out as if nothing had been guessed. Inside the tree, everything is conditional on the guesses above it, but it turns out this isn't an issue.

```

rup 1 ~x2ge2 >= 1 ;      if the guess were part of the problem
rup x1eq0 ==> 1 ~x2ge2 >= 1 ;    what we actually write

```

- Whatever we can prove without assumptions we can prove with them, using the same rules in the same order, for a constant amount of extra work per step.
- For **rup**, put the assumptions on the line. Negating the implication makes the checker set every one of them, leaving exactly the problem we knew how to refute.
- For **pol**, do nothing whatsoever: add two conditional constraints and their conditions add too, divide and the condition divides with it. The assumptions come along for free.

Matthew McIlree's thesis again, chapter 2, where the division and saturation are done properly.

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$V \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	% Guesses took these away
$W \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$X \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$Y \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$Z \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$V \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	% Guesses took these away
$W \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$X \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$Y \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	
$Z \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$V \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$
 $W \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$
 $X \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$
 $Y \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$
 $Z \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$

$w_{=1} + w_{=2} + w_{=3} + w_{=4} + w_{=5} \geq 1$

% Guesses took these away

% W takes some value

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$V \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$						% Guesses took these away
$W \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$w=1 +$	$w=2 +$	$w=3 +$	$w=4 +$	$w=5 \geq$	1 % W takes some value
$X \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$x=1 +$	$x=2 +$	$x=3 +$	$x=4 +$	$x=5 \geq$	1 % X takes some value
$Y \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$y=1 +$	$y=2 +$	$y=3 +$	$y=4 +$	$y=5 \geq$	1 % Y takes some value
$Z \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$z=1 +$	$z=2 +$	$z=3 +$	$z=4 +$	$z=5 \geq$	1 % Z takes some value

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$V \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$						% Guesses took these away
$W \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$w=1 +$	$w=2 +$	$w=3 +$	$w=4 +$	$w=5 \geq$	1 % W takes some value
$X \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$x=1 +$	$x=2 +$	$x=3 +$	$x=4 +$	$x=5 \geq$	1 % X takes some value
$Y \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$y=1 +$	$y=2 +$	$y=3 +$	$y=4 +$	$y=5 \geq$	1 % Y takes some value
$Z \in \{1 \text{ } 2 \text{ } 3 \text{ } 4 \text{ } 5\}$	$z=1 +$	$z=2 +$	$z=3 +$	$z=4 +$	$z=5 \geq$	1 % Z takes some value

$\rightarrow$	$-v=1 + -w=1 + -x=1 + -y=1 + -z=1 \geq -1$	% At most one variable = 1
$\rightarrow$	$-v=2 + -w=2 + -x=2 + -y=2 + -z=2 \geq -1$	% At most one variable = 2
$\rightarrow$	$-v=3 + -w=3 + -x=3 + -y=3 + -z=3 \geq -1$	% At most one variable = 3

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$$\begin{array}{llllll}
 V \in \{1 & 2 & 3 & 4 & 5\} & & \% \text{ Guesses took these away} \\
 W \in \{1 & 2 & 3 & 4 & 5\} & w_{=1} + w_{=2} + w_{=3} + w_{=4} + w_{=5} \geq 1 & \% W \text{ takes some value} \\
 X \in \{1 & 2 & 3 & 4 & 5\} & x_{=1} + x_{=2} + x_{=3} + x_{=4} + x_{=5} \geq 1 & \% X \text{ takes some value} \\
 Y \in \{1 & 2 & 3 & 4 & 5\} & y_{=1} + y_{=2} + y_{=3} + y_{=4} + y_{=5} \geq 1 & \% Y \text{ takes some value} \\
 Z \in \{1 & 2 & 3 & 4 & 5\} & z_{=1} + z_{=2} + z_{=3} + z_{=4} + z_{=5} \geq 1 & \% Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + -x_{=1} + -y_{=1} + -z_{=1} \geq -1 \quad \% \text{ At most one variable} = 1 \\
 \rightarrow & -v_{=2} + -w_{=2} + -x_{=2} + -y_{=2} + -z_{=2} \geq -1 \quad \% \text{ At most one variable} = 2 \\
 \rightarrow & -v_{=3} + -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \quad \% \text{ At most one variable} = 3
 \end{array}$$

$$w_{=4} + w_{=5} + x_{=4} + x_{=5} + y_{=4} + y_{=5} + z_{=4} + z_{=5} - v_{=1} - v_{=2} - v_{=3} \geq 1 \quad \% \text{ Sum all constraints so far}$$

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$$\begin{array}{llllll}
 V \in \{1 & 2 & 3 & 4 & 5\} & & \% \text{ Guesses took these away} \\
 W \in \{1 & 2 & 3 & 4 & 5\} & w_{=1} + w_{=2} + w_{=3} + w_{=4} + w_{=5} \geq 1 & \% W \text{ takes some value} \\
 X \in \{1 & 2 & 3 & 4 & 5\} & x_{=1} + x_{=2} + x_{=3} + x_{=4} + x_{=5} \geq 1 & \% X \text{ takes some value} \\
 Y \in \{1 & 2 & 3 & 4 & 5\} & y_{=1} + y_{=2} + y_{=3} + y_{=4} + y_{=5} \geq 1 & \% Y \text{ takes some value} \\
 Z \in \{1 & 2 & 3 & 4 & 5\} & z_{=1} + z_{=2} + z_{=3} + z_{=4} + z_{=5} \geq 1 & \% Z \text{ takes some value} \\
 \\
 \rightarrow & -v_{=1} + -w_{=1} + -x_{=1} + -y_{=1} + -z_{=1} \geq -1 & \% \text{ At most one variable} = 1 \\
 \rightarrow & -v_{=2} + -w_{=2} + -x_{=2} + -y_{=2} + -z_{=2} \geq -1 & \% \text{ At most one variable} = 2 \\
 \rightarrow & -v_{=3} + -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 & \% \text{ At most one variable} = 3 \\
 \\
 w_{=4} + w_{=5} + x_{=4} + x_{=5} + y_{=4} + y_{=5} + z_{=4} + z_{=5} - v_{=1} - v_{=2} - v_{=3} \geq 1 & \% \text{ Sum all constraints so far} \\
 v_{=1} + v_{=2} + v_{=3} \geq 0 & \% \text{ Each } v_{=i} \text{ non-negative}
 \end{array}$$

Justifying All-Different Failures, With Assumptions

Same argument, same **pol**, except the **values the guesses took away** are still written down.

$$\begin{array}{llllll}
 V \in \{1 & 2 & 3 & 4 & 5\} & & \% \text{ Guesses took these away} \\
 W \in \{1 & 2 & 3 & 4 & 5\} & w_{=1} + w_{=2} + w_{=3} + w_{=4} + w_{=5} \geq 1 & \% W \text{ takes some value} \\
 X \in \{1 & 2 & 3 & 4 & 5\} & x_{=1} + x_{=2} + x_{=3} + x_{=4} + x_{=5} \geq 1 & \% X \text{ takes some value} \\
 Y \in \{1 & 2 & 3 & 4 & 5\} & y_{=1} + y_{=2} + y_{=3} + y_{=4} + y_{=5} \geq 1 & \% Y \text{ takes some value} \\
 Z \in \{1 & 2 & 3 & 4 & 5\} & z_{=1} + z_{=2} + z_{=3} + z_{=4} + z_{=5} \geq 1 & \% Z \text{ takes some value}
 \end{array}$$

$$\begin{array}{ll}
 \rightarrow & -v_{=1} + -w_{=1} + -x_{=1} + -y_{=1} + -z_{=1} \geq -1 \quad \% \text{ At most one variable} = 1 \\
 \rightarrow & -v_{=2} + -w_{=2} + -x_{=2} + -y_{=2} + -z_{=2} \geq -1 \quad \% \text{ At most one variable} = 2 \\
 \rightarrow & -v_{=3} + -w_{=3} + -x_{=3} + -y_{=3} + -z_{=3} \geq -1 \quad \% \text{ At most one variable} = 3
 \end{array}$$

$$\begin{array}{ll}
 w_{=4} + w_{=5} + x_{=4} + x_{=5} + y_{=4} + y_{=5} + z_{=4} + z_{=5} - v_{=1} - v_{=2} - v_{=3} \geq & 1 \quad \% \text{ Sum all constraints so far} \\
 v_{=1} + v_{=2} + v_{=3} \geq & 0 \quad \% \text{ Each } v_{=i} \text{ non-negative}
 \end{array}$$

$$w_{=4} + w_{=5} + x_{=4} + x_{=5} + y_{=4} + y_{=5} + z_{=4} + z_{=5} \geq 1 \quad \% \text{ Sum above two constraints}$$

Nothing in the **pol** knew about assumptions: the guesses the failure didn't need have cancelled themselves out, and what is left is false under the ones that remain, so it reads $0 \geq 1$.

Reasons, Explanations, and Lazy Clause Generation

Every inference **kindling** logs is guarded by **every guess** made on the way to the node. Always legal, because between them the guesses determine the whole node—and often more than was needed.

Reasons, Explanations, and Lazy Clause Generation

Every inference **kindling** logs is guarded by **every guess** made on the way to the node. Always legal, because between them the guesses determine the whole node—and often more than was needed.

- A real solver could state a **reason**: the handful of literals the propagator actually looked at.

Reasons, Explanations, and Lazy Clause Generation

Every inference **kindling** logs is guarded by **every guess** made on the way to the node. Always legal, because between them the guesses determine the whole node—and often more than was needed.

- A real solver could state a **reason**: the handful of literals the propagator actually looked at.
- That clause is the same object a lazy clause generation solver has to produce to explain a propagation.

Reasons, Explanations, and Lazy Clause Generation

Every inference **kindling** logs is guarded by **every guess** made on the way to the node. Always legal, because between them the guesses determine the whole node—and often more than was needed.

- A real solver could state a **reason**: the handful of literals the propagator actually looked at.
- That clause is the same object a lazy clause generation solver has to produce to explain a propagation.
- So a solver that already does lazy clause generation is most of the way to logging proofs.

Reasons, Explanations, and Lazy Clause Generation

Every inference **kindling** logs is guarded by **every guess** made on the way to the node. Always legal, because between them the guesses determine the whole node—and often more than was needed.

- A real solver could state a **reason**: the handful of literals the propagator actually looked at.
- That clause is the same object a lazy clause generation solver has to produce to explain a propagation.
- So a solver that already does lazy clause generation is most of the way to logging proofs.
- There is one catch: a reason can be **wrong**. This will make a proof fail, but can be hard to debug. . .

Deletions

In **Kindling**, every constraint a proof derives joins the checker's database and stays there.

- **VeriPB** has a **del** rule. **Glasgow** tags every constraint it derives with the depth it derived it at, and when the search backtracks past that depth, deletes the lot in one command. Doing that here would leave it holding around
- As well as saving memory, **rup** and **red** can be a lot faster over a small active database.
- Deletion also makes you ask what is worth keeping. Look back at the proof with nothing asserted: the all-different **pol** is derived **twice**, identically, once in each branch—and it is unconditional, so like the **@atleast** lines above it, it could have been derived once at the top and cited from both.

What Constraints Can We Do?

Glasgow has thirty-seven constraints, and every inference every one of them makes is justified:
Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede.

Also reformulations, each of which infers something the model never said, and **derives** it rather than asserting it:

What Constraints Can We Do?

Glasgow has thirty-seven constraints, and every inference every one of them makes is justified:
Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede.

Also reformulations, each of which infers something the model never said, and **derives** it rather than asserting it:

- Auto-tabulation: notice that a subproblem is small, solve it, and replace it by a table.

What Constraints Can We Do?

Glasgow has thirty-seven constraints, and every inference every one of them makes is justified: Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede.

Also reformulations, each of which infers something the model never said, and **derives** it rather than asserting it:

- Auto-tabulation: notice that a subproblem is small, solve it, and replace it by a table.
- Capacity strengthening by integrality, conflict cliques across resources, lifted cover cuts: three ways of telling a scheduling problem something about itself that none of its constraints says.

What Constraints Can We Do?

Glasgow has thirty-seven constraints, and every inference every one of them makes is justified: Abs, AllDifferent, AllEqual, Among, AtMostOne, BinPacking, Circuit, Comparison, Count, Cumulative, Diffn, Disjunctive, DivMod, Element, Equals, GCC, In, Increasing, Inverse, Knapsack, Lex, Linear, Logical, MDD, MinDistance, MinMax, Multiply, NValue, Parity, PlusMinus, Power, Regular, SeqPrecedeChain, SmartTable, Sort, Table, ValuePrecede.

Also reformulations, each of which infers something the model never said, and **derives** it rather than asserting it:

- Auto-tabulation: notice that a subproblem is small, solve it, and replace it by a table.
- Capacity strengthening by integrality, conflict cliques across resources, lifted cover cuts: three ways of telling a scheduling problem something about itself that none of its constraints says.
- Difference logic: spot that a pile of two-term inequalities is a precedence network, and hand it to a temporal propagator.

VeriPB Justifications Bag of Tricks

- **Search you did not have to do.** A table propagator finds an unsupported value efficiently, not by trying it. The justification says “had you guessed this, it would not have worked”.
- **Case analysis.** If we can show something holds under a complete set of cases, it holds generally.
- **Linear arithmetic.** Add constraints, scale them, round them off. Linear, Abs.
- **Counting.** Pigeonhole, and recovering an at-most-one from pairwise conflicts. All-different’s Hall sets, GCC, NValue, Among.
- **Naming things.** Introduce a flag standing for a condition and argue about that instead. Reification, Cumulative’s before and after flags, Smart Table.
- **Dynamic programming.** A flag per state, an implication per transition: replay the computation rather than arguing about its answer. Regular, Knapsack, Decision Diagrams.
- **Orderings and reachability.** A rank witness, or a transitive closure. Sort, Inverse, Circuit.
- **Flow arguments.** Theoretically possible, unclear if they’re practical yet.

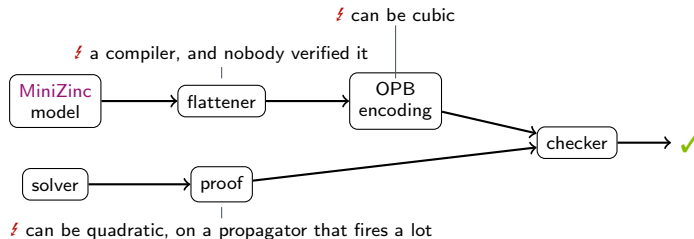
Every propagator on the last slide needs two or three of these put together cleverly.

Multiplication: Doable, But Not Pretty

Multiplication, division and modulus are also possible. It's fairly horrible, though.

- A product is awkward in **both** views at once. The bits do not add up into anything the order literals can see, and a bound on a product is not a bound on anything linear, so neither of the two things that rescue everything else is available.
- What comes out is bit-level case analysis and a long, specific derivation per propagation rule, rather than one recipe you could explain in a slide.
- We do not know whether that ugliness is **inherent**. We don't have a proof that we can't do a lot better.

Omissions, Lies, and Future Work



And all of it only carries integers: sets, unbounded integers, floats and strings have no encoding at all.

Open Problems: The Other Four Hundred Propagators

The joke used to be that we had done six constraints, and the global constraint catalogue had about four hundred.

- That is getting harder to say with a straight face. Scheduling was the large hole, and it is mostly filled: time-tabling, edge finding, energetic reasoning, not-first and not-last, over variable durations, demands and capacities.
- None of those needed anything added to the proof vocabulary. What they needed was somebody to sit down and work out the argument, which is a different kind of problem from an open one.
- The techniques generalise. Anything that can be written as a smart table or MDD is already justifiable, which quietly covers a good fraction of the catalogue without anyone doing it one at a time.

Open Problems: Everything That Is Not Finite Domain Integers

- **Set variables.** Can represent sets by lots of Booleans, but solvers have more efficient alternatives.
- **Unbounded integers.** Our encoding opens by asking how many bits a variable needs, and a variable with no bounds has no answer.
- **Floating point.** Even the semantics are hairy.
- **Strings.** Again, needs a size bound, which can be hard to trust.

The pattern is that we have proof logging for the parts of constraint programming that look like integers, and that is not all of constraint programming.

Open Problems: Efficiency and Scalability

Two problems make it hard to apply this to some industrial instances:

- **Encoding size.** Our encoding procedures involve a lot of “copy-paste” loops. A cubic encoding gets annoying quickly.
- **Argument size.** The same thing can happen with some justifications. Here even quadratic can be annoying on a frequently-firing propagator.

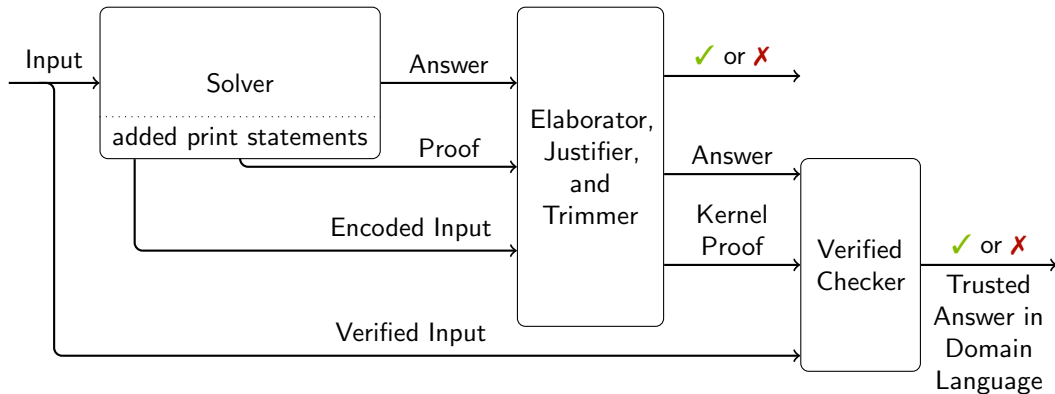
Better Performance by Generating and Checking Less of the Proof?

One potential approach, with ideas stolen from DRCP and the Pumpkin solver (Flippo, Sidorov, Marijnissen, Smits and Demirović, CP 2024):

- The solver emits **a** lines and nothing else, but fills in the **reason** and the hints in the annotation fields.
- **Trim first**. Elaborate the skeleton, throw away every inference the conclusion turned out not to need, and only then start working out justifications.
- Justify **outside** the solver. Every derivation in this talk needs only the reason, plus occasionally something small like an automaton's state, so a justifier can do it afterwards rather than every solver doing it for itself.
- When nothing in the library fits: fall back on a decomposition, an LP relaxation, or handing the goal to a proof logging pseudo-Boolean or SAT solver to find the argument for you.

Coming soon: a production-quality trimmer and justifier for **VeriPB** that can actually do this.

Open Problems: The Encodings Gap



Open Problems: The Encodings Gap

- We need to get the OPB encoding from somewhere. We can create it from the solver's input, but you need to trust the process.
- Nobody writes their model in the solver's own input format. They write **MiniZinc** or **CPMpy**, and something flattens it.
- A flattener is a **compiler**. It reformulates, it decomposes, it introduces variables, it is thousands of lines long, and it has not been verified.

Open Problems: The Encodings Gap

Very recent work: s-expression CP, which the verified checker reads. $SEND + MORE = MONEY$:

```
(
  (version 1)
  (variables (s 1 9) (e 0 9) (n 0 9) (d 0 9) (m 1 9) (o 0 9) (r 0 9) (y 0 9))
  (constraints
    (money_a_d all_different (s e n d m o r y))
    (money_sum lin_equals (1000 s 100 e 10 n 1 d 1000 m 100 o 10 r 1 e
                          -10000 m -1000 o -100 n -10 e -1 y) 0))
  (prob_type enumerate)
)
```

- Every variable and every constraint, written out longhand. No loops, no conditionals, no arrays, no parameters, no separate data file.
- What it does have is **meaning**: forty-odd constraint types whose semantics are formalised inside the verified checker—all-different, cumulative, circuit, regular, knapsack, etc.
- The checker re-derives the pseudo-Boolean encoding from this file, and the proof is checked against **that**. If you trust the SCP file, you can trust the output.
- Can compile MiniZinc to SCP, but not in a verified way.

Proof Logging World Domination Plans

- Constraint programming solvers are being used autonomously to make important decisions about lives and livelihoods. Undetected wrong answers are a real problem.

Proof Logging World Domination Plans

- Constraint programming solvers are being used autonomously to make important decisions about lives and livelihoods. Undetected wrong answers are a real problem.
- You can't do useful scientific comparisons if solvers can randomly cheat.

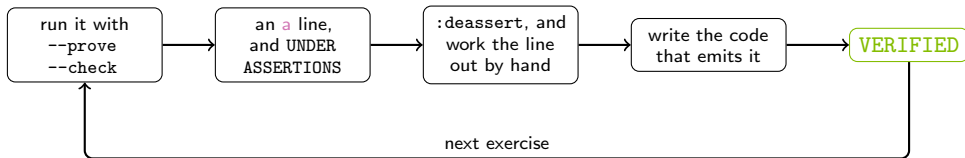
Proof Logging World Domination Plans

- Constraint programming solvers are being used autonomously to make important decisions about lives and livelihoods. Undetected wrong answers are a real problem.
- You can't do useful scientific comparisons if solvers can randomly cheat.
- Once you know how to do it, proof logging speeds up development.

Proof Logging World Domination Plans

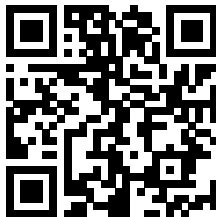
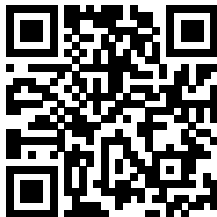
- Constraint programming solvers are being used autonomously to make important decisions about lives and livelihoods. Undetected wrong answers are a real problem.
- You can't do useful scientific comparisons if solvers can randomly cheat.
- Once you know how to do it, proof logging speeds up development.
- The SAT community has already done this. Bugs used to be a big problem. Now, proofs are mandatory, and bugs are (mostly) not a problem. We all know CP is better than SAT, so what's our excuse?

Exercises



Four times round: **rup** for table, **pol** for coefficients and for Hall sets—and then `--bug`, to break a propagator on purpose.

Code for the Exercises



Kindling Solver, see README.md and files in `slides-proofs/`:

`https://github.com/ciaranm/kindling`

VeriPB (official version), see README.md and try to run `veripb justified.opb justified.pbp`:

`https://gitlab.com/MIA0research/software/VeriPB`

VeriPB (experimental interactive mode, probably Linux and Mac only), see README.md and then use `cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml`:

`https://github.com/ciaranm/veripb-repl`

Installing Kindling

```
git clone https://github.com/ciaranm/kindling.git
cd kindling
python3 -m kindling pigeonhole --prove pigeonholeproof --check
python3 -m unittest discover -s tests -t .
```

Installing Kindling

```
git clone https://github.com/ciaranm/kindling.git
cd kindling
python3 -m kindling pigeonhole --prove pigeonholeproof --check
python3 -m unittest discover -s tests -t .
```

- Nothing to install. Python's standard library, and the VeriPB you already have.

Installing Kindling

```
git clone https://github.com/ciaranm/kindling.git
cd kindling
python3 -m kindling pigeonhole --prove pigeonholeproof --check
python3 -m unittest discover -s tests -t .
```

- Nothing to install. Python's standard library, and the VeriPB you already have.
- On a fresh clone the only failures are in tests/test_exercises.py, one per exercise, each naming what is still unjustified. If anything else fails you have broken the solver rather than not finished it.

Installing Kindling

```
git clone https://github.com/ciaranm/kindling.git
cd kindling
python3 -m kindling pigeonhole --prove pigeonholeproof --check
python3 -m unittest discover -s tests -t .
```

- Nothing to install. Python's standard library, and the VeriPB you already have.
- On a fresh clone the only failures are in tests/test_exercises.py, one per exercise, each naming what is still unjustified. If anything else fails you have broken the solver rather than not finished it.
- The worksheet is docs/practical.md, and it has more detail than these slides. The answers are on the ash branch—there is nothing to cheat at, so go and look if you are stuck.

Installing Kindling

```
git clone https://github.com/ciaranm/kindling.git
cd kindling
python3 -m kindling pigeonhole --prove pigeonholeproof --check
python3 -m unittest discover -s tests -t .
```

- Nothing to install. Python's standard library, and the VeriPB you already have.
- On a fresh clone the only failures are in tests/test_exercises.py, one per exercise, each naming what is still unjustified. If anything else fails you have broken the solver rather than not finished it.
- The worksheet is docs/practical.md, and it has more detail than these slides. The answers are on the ash branch—there is nothing to cheat at, so go and look if you are stuck.
- One switch worth knowing before you start: --no-justifications logs the search and leaves out everything the propagators worked out. Try it on agreement and then on tight-sum.

VeriPB's Brand New Experimental Interactive Mode

```
cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml
```

Type a line, and find out what it did:

```
pbp> pol @lin1 @x2ge3_up + @x3ge3_up + @x1ge4_up + s ;  
pbp> :show x1ge4  
pbp> :undo  
(line rejected, state unchanged)
```

VeriPB's Brand New Experimental Interactive Mode

```
cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml
```

Type a line, and find out what it did:

```
pbp> pol @lin1 @x2ge3_up + @x3ge3_up + @x1ge4_up + s ;  
pbp> :show x1ge4  
pbp> :undo  
(line rejected, state unchanged)
```

- `:show` lists the constraint database, filtered by variable or by number. `:list` prints what you have typed so far, `:undo` takes it back, and `:save` writes the session out as a real `.pbp`.

VeriPB's Brand New Experimental Interactive Mode

```
cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml
```

Type a line, and find out what it did:

```
pbp> pol @lin1 @x2ge3_up + @x3ge3_up + @x1ge4_up + s ;  
pbp> :show x1ge4  
pbp> :undo  
(line rejected, state unchanged)
```

- `:show` lists the constraint database, filtered by variable or by number. `:list` prints what you have typed so far, `:undo` takes it back, and `:save` writes the session out as a real `.pbp`.
- A line that does not check is rejected and the state is left alone, so you can guess and keep guessing. Which is a different activity from editing Python, rerunning the solver, and reading a rejection at line 349.

VeriPB's Brand New Experimental Interactive Mode

cargo run -p veripb-repl -m path-to-veripb-repl/Cargo.toml
Type a line, and find out what it did:

```
pbp> pol @lin1 @x2ge3_up + @x3ge3_up + @x1ge4_up + s ;
pbp> :show x1ge4
pbp> :undo
(line rejected, state unchanged)
```

- `:show` lists the constraint database, filtered by variable or by number. `:list` prints what you have typed so far, `:undo` takes it back, and `:save` writes the session out as a real `.pbp`.
- A line that does not check is rejected and the state is left alone, so you can guess and keep guessing. Which is a different activity from editing Python, rerunning the solver, and reading a rejection at line 349.
- It is genuinely brand new and it is on a branch. If it is not behaving on the day, then `--trace-failed` and `--trace-pol` do a coarser version of the same job.

Do the Derivation Before You Write the Code

:deassert takes the first **a** line in the proof, deletes it, and lets you type what should have been there instead:

```
pbp> :deassert
Line 14: a x1eq0 ==> >= 1 : : all_different_hall : x2, x3, x4 in {0, 1} ;
edit> pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
      ConstraintId 55: 1 x2eq2 1 x3eq2 1 x4eq2 >= 1
edit> :done
```

Do the Derivation Before You Write the Code

:deassert takes the first **a** line in the proof, deletes it, and lets you type what should have been there instead:

```
pbp> :deassert
Line 14: a x1eq0 ==> >= 1 : : all_different_hall : x2, x3, x4 in {0, 1} ;
edit> pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
      ConstraintId 55: 1 x2eq2 1 x3eq2 1 x4eq2 >= 1
edit> :done
```

- Every exercise is this, and then the code that does it for you. The hard half is working the line out, and the solver is no help with that at all.

Do the Derivation Before You Write the Code

:deassert takes the first **a** line in the proof, deletes it, and lets you type what should have been there instead:

```
pbp> :deassert
Line 14: a x1eq0 ==> >= 1 : : all_different_hall : x2, x3, x4 in {0, 1} ;
edit> pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
      ConstraintId 55: 1 x2eq2 1 x3eq2 1 x4eq2 >= 1
edit> :done
```

- Every exercise is this, and then the code that does it for you. The hard half is working the line out, and the solver is no help with that at all.
- It prints **what you derived**, which is the only way to tell a sum that is right from one that is merely accepted. :done then replays the rest of the proof on top, so you find out at once whether everything after it still follows.

Do the Derivation Before You Write the Code

:deassert takes the first **a** line in the proof, deletes it, and lets you type what should have been there instead:

```
pbp> :deassert
Line 14: a x1eq0 ==> >= 1 : : all_different_hall : x2, x3, x4 in {0, 1} ;
edit> pol @amo1_0 @amo1_1 + @atleast2 + @atleast3 + @atleast4 + ;
      ConstraintId 55: 1 x2eq2 1 x3eq2 1 x4eq2 >= 1
edit> :done
```

- Every exercise is this, and then the code that does it for you. The hard half is working the line out, and the solver is no help with that at all.
- It prints **what you derived**, which is the only way to tell a sum that is right from one that is merely accepted. :done then replays the rest of the proof on top, so you find out at once whether everything after it still follows.
- Two :deasserts and that proof is VERIFIED, with the solver untouched. Then go and write the propagator, knowing what it has to emit.

Exercise 1: Table

`table_int.py` removes a value when no tuple can support it any more, and asserts that it was allowed to. It was allowed to, and the reason is reverse unit propagation.

Replace the two `Assert(...)` with `Rup()` and make `two-tables` verify.

```
python3 -m kindling tight-sum --prove tight-sum-proof --check
python3 -m kindling too-small --prove too-small-proof --check
```

Exercise 1: Table

`table_int.py` removes a value when no tuple can support it any more, and asserts that it was allowed to. It was allowed to, and the reason is reverse unit propagation.

Replace the two `Assert(...)` with `Rup()` and make `two-tables` verify.

That is one word, so spend the time you saved on this. Try exactly the same thing in `int_lin_le`: replace its `Pol(...)` with `Rup()`, and run

```
python3 -m kindling tight-sum --prove tight-sum-proof --check
python3 -m kindling too-small --prove too-small-proof --check
```

Exercise 1: Table

`table_int.py` removes a value when no tuple can support it any more, and asserts that it was allowed to. It was allowed to, and the reason is reverse unit propagation.

Replace the two `Assert(...)` with `Rup()` and make `two-tables` verify.

That is one word, so spend the time you saved on this. Try exactly the same thing in `int_lin_le`: replace its `Pol(...)` with `Rup()`, and run

```
python3 -m kindling tight-sum --prove tight-sum-proof --check
python3 -m kindling too-small --prove too-small-proof --check
```

One of those still verifies and the other does not. Work out why before you move on: it is the reason the next two exercises are harder than this one, and it is a property of how integers are encoded rather than anything about tables or sums.

Exercise 2: Negative Coefficients

`int_lin_le` justifies a bound with cutting planes: the constraint saying what the constraint is, plus one channelling constraint per term. Every bit cancels, and what is left is the clause.

The reference handles positive coefficients. A negative one needs the same sum with **one thing** changed, because a term that counts downwards needs its variable bounded from the other side. Look at channelling, and at the line defining every variable's order literals:

```
@x1ge3_up @x1ge3_dn x1ge3 <==> 4 x1b2 2 x1b1 1 x1b0 >= 3 ;
```

One line, two constraints, a label for each. What you add is what **VeriPB** stored:

```
@x1ge3_up 4 x1b2 2 x1b1 1 x1b0 3 ~x1ge3 >= 3 ;
@x1ge3_dn 4 ~x1b2 2 ~x1b1 1 ~x1b0 5 x1ge3 >= 5 ;
```

Make over-budget verify.

Exercise 2: Negative Coefficients

`int_lin_le` justifies a bound with cutting planes: the constraint saying what the constraint is, plus one channelling constraint per term. Every bit cancels, and what is left is the clause.

The reference handles positive coefficients. A negative one needs the same sum with **one thing** changed, because a term that counts downwards needs its variable bounded from the other side. Look at channelling, and at the line defining every variable's order literals:

```
@x1ge3_up @x1ge3_dn x1ge3 <==> 4 x1b2 2 x1b1 1 x1b0 >= 3 ;
```

One line, two constraints, a label for each. What you add is what **VeriPB** stored:

```
@x1ge3_up 4 x1b2 2 x1b1 1 x1b0 3 ~x1ge3 >= 3 ;
@x1ge3_dn 4 ~x1b2 2 ~x1b1 1 ~x1b0 5 x1ge3 >= 5 ;
```

Make over-budget verify.

Getting this wrong gives you REJECTED rather than UNDER ASSERTIONS, so you will know.

Exercise 3: Hall Sets

`all_different_int` never removes a value—it only fails, when it finds a set of variables with fewer values between them than there are variables. That argument is not unit propagation. It needs adding constraints up.

Write `hall_steps`, and swap the `Assert` for `Pol(self.hall_steps(variables, values))`. The constraints you have:

- `@amo{c}_{v}`: for each value, at most one of these variables takes it.
- `@atleast{i}`: each variable takes at least one value. Derived in the preamble rather than asserted in the `.opb`, because it is a consequence of what the literals mean rather than something the model says.

Make `pigeonhole` and `squeeze` verify.

Exercise 3: Hall Sets

`all_different_int` never removes a value—it only fails, when it finds a set of variables with fewer values between them than there are variables. That argument is not unit propagation. It needs adding constraints up.

Write `hall_steps`, and swap the `Assert` for `Pol(self.hall_steps(variables, values))`. The constraints you have:

- `@amo{c}_{v}`: for each value, at most one of these variables takes it.
- `@atleast{i}`: each variable takes at least one value. Derived in the preamble rather than asserted in the `.opb`, because it is a consequence of what the literals mean rather than something the model says.

Make `pigeonhole` and `squeeze` verify.

You will also want `w`, which weakens a constraint by dropping a term. Work out what needs dropping, and why the checker accepts the proof without it anyway.

Exercise 4: Debugging

Now the point of all of it. `--bug` breaks a propagator on purpose:

```
python3 -m kindling puzzle --bug table-forgets-a-tuple --prove bug --check
```

Run all four bugs against `puzzle`, `latin`, `tight-sum` and `over-budget`, and fill in what happens. Some of it is not what you would guess.

Exercise 4: Debugging

Now the point of all of it. `--bug` breaks a propagator on purpose:

```
python3 -m kindling puzzle --bug table-forgets-a-tuple --prove bug --check
```

Run all four bugs against `puzzle`, `latin`, `tight-sum` and `over-budget`, and fill in what happens. Some of it is not what you would guess.

- Three of them make the solver **lose the only solution** and report that there is none. No test of the answer alone would have caught that, on an instance whose answer you did not already know.

Exercise 4: Debugging

Now the point of all of it. `--bug` breaks a propagator on purpose:

```
python3 -m kindling puzzle --bug table-forgets-a-tuple --prove bug --check
```

Run all four bugs against `puzzle`, `latin`, `tight-sum` and `over-budget`, and fill in what happens. Some of it is not what you would guess.

- Three of them make the solver **lose the only solution** and report that there is none. No test of the answer alone would have caught that, on an instance whose answer you did not already know.
- One leaves the solver completely correct. Right answer, every time, and only the derivation is wrong. Nothing you could write a test for is broken. The proof still fails.

Exercise 4: Debugging

Now the point of all of it. `--bug` breaks a propagator on purpose:

```
python3 -m kindling puzzle --bug table-forgets-a-tuple --prove bug --check
```

Run all four bugs against `puzzle`, `latin`, `tight-sum` and `over-budget`, and fill in what happens. Some of it is not what you would guess.

- Three of them make the solver **lose the only solution** and report that there is none. No test of the answer alone would have caught that, on an instance whose answer you did not already know.
- One leaves the solver completely correct. Right answer, every time, and only the derivation is wrong. Nothing you could write a test for is broken. The proof still fails.
- And the one worth arguing about: run an unsound bug against an instance that really has no solution. The proof **verifies**. Work out why that is the correct behaviour rather than a hole.

<https://ciaranm.github.io/>

ciaran.mccreesh@glasgow.ac.uk

